

Regime-Aware Machine Learning for Equity Trading

An Event-Driven Approach with Technical Pattern Detection,
Triple-Barrier Labeling, and Walk-Forward Validation

Zeineb Turki

Supervisor: Prof. Dr. Hadházi Dániel

Budapest University of Technology and Economics

2026

May 2026

Assignment Statement

This page is reserved for the official assignment statement issued by Budapest University of Technology and Economics. The signed original document is to be included in the printed submission.

The student is tasked with designing and implementing a machine-learning pipeline for equity trading that leverages technical chart-pattern detection, triple-barrier labeling, and walk-forward cross-validation. The system must be evaluated on historical S&P 500 (SPY) data spanning at least ten years, with thorough analysis of classification accuracy, financial performance, and generalization behaviour.

Signed: _____

Date: _____

Table of Contents

1. Abstract

2. Introduction

- 2.1 Why Financial Prediction is Hard
- 2.2 Event-Based Learning
- 2.3 Research Questions
- 2.4 Contributions

3. Background and Related Work

- 3.1 Technical Analysis and Market Psychology
- 3.2 Technical Indicators
- 3.3 Triple-Barrier Labeling
- 3.4 Ensemble Tree Methods
- 3.5 Walk-Forward Validation
- 3.6 Related Work

4. System Design

- 4.1 Architecture
- 4.2 Module Structure
- 4.3 Design Principles
- 4.4 Design Decisions

5. Data

- 5.1 Dataset
- 5.2 Cleaning
- 5.3 Indicator Computation

6. Pattern Detection

- 6.1 Support and Resistance
- 6.2 Channels
- 6.3 Triangles
- 6.4 Multiple Tops and Bottoms
- 6.5 Touch Events
- 6.6 Detection Summary

7. Case Studies

- 7.1 Support/Resistance Event Case Study
- 7.2 Multiple Top Case Study

8. Feature Engineering

- 8.1 Feature Groups
- 8.2 Leakage Prevention
- 8.3 Feature Importance

9. Triple-Barrier Labeling

- 9.1 Method
- 9.2 Label Distribution

9.3 Parameters as Hyperparameters

10. Machine Learning Models

10.1 Model Selection

10.2 Why Trees

10.3 Tree Diagnostics

11. Validation

11.1 Chronological Split

11.2 Walk-Forward Cross-Validation

11.3 Standard K-Fold

11.4 Why Multiple Validation Strategies

12. Trading Simulation

12.1 Motivation

12.2 Simulation Mechanics

12.3 Performance Metrics

12.4 Assumptions and Limitations

13. Hyperparameter Optimization

13.1 Search Space

13.2 Procedure

13.3 Results

13.4 Overfitting Risk

14. Experimental Results

14.1 Best Parameters

14.2 Model Comparison

14.3 Equity Curve

14.4 Touch Events Analysis

14.5 Confusion Matrix

15. Generalization Analysis

15.1 Walk-Forward Variance

15.2 Walk-Forward vs. K-Fold

15.3 Sources of Variance

15.4 Confidence Interpretation

16. F-Beta Analysis

16.1 False Positive / False Negative Asymmetry

16.2 Results

16.3 Implications

17. Discussion

17.1 Classification vs. Profitability

17.2 Touch Events

17.3 Leakage Prevention

17.4 F-Beta and Trading Objectives

17.5 Strengths

17.6 Weaknesses

17.7 Noise and Uncertainty in Finance

18. Limitations

19. Future Work

20. Conclusion

Bibliography

A. Appendix A: Module Overview

B. Appendix B: Parameter Reference

C. Appendix C: Notebook Guide and Reproducibility

1. Abstract

This thesis presents a regime-aware machine-learning pipeline for equity trading on the S&P 500 ETF (SPY). Rather than training on every market bar, the system first identifies structurally meaningful events through four technical-pattern detectors, support/resistance levels, price channels, triangle formations, and multiple tops/bottoms, reducing 4,023 daily bars to 132 high-signal events. Each event is enriched with 48 features spanning momentum, volatility, trend, and pattern geometry, then labeled using the triple-barrier method of Lopez de Prado (2018), which encodes realistic trade outcomes (profit target, stop loss, or maximum holding period) rather than naive directional predictions.

A Random Forest classifier is trained and evaluated through three complementary validation strategies: a chronological train/test split, walk-forward cross-validation with four expanding-window folds, and standard k-fold cross-validation. A grid search over 100 configurations jointly optimizes pattern-detection and labeling hyperparameters, revealing that classification accuracy and trading profitability are partially decoupled: the best F1 configuration (pt=2.0, sl=1.5, mh=10, F1=0.569) differs from the most profitable one (pt=2.5, sl=3.0, mh=20, return=25.9%). Walk-forward testing yields an F1 of 0.282 ± 0.008 with a 52.3% win rate and a Sharpe ratio of 0.131, while the baseline random classifier achieves only 0.160 F1 and 28.6% accuracy. An F-beta analysis reveals that F2 (0.299) slightly outperforms F1 (0.282), suggesting the model captures more value when recall is weighted higher. Although the system demonstrates measurable alpha over random trading, the high variance across walk-forward folds highlights the inherent difficulty of generalizing pattern-based strategies across market regimes.

2. Introduction

2.1 Why Financial Prediction is Hard

Financial markets represent one of the most challenging prediction domains in applied machine learning. Unlike image recognition, where a cat remains a cat regardless of when the photo was taken, financial time series are fundamentally non-stationary: the statistical properties of returns shift as macroeconomic regimes change, monetary policy evolves, and market participants adapt their strategies. The Efficient Market Hypothesis (EMH), in its semi-strong form, posits that prices already incorporate all publicly available information, rendering systematic technical prediction impossible in theory. While the EMH has been challenged by decades of behavioral finance research, the core insight remains practically relevant: any exploitable pattern, once widely known, tends to be arbitrated away.

The signal-to-noise ratio in daily equity returns is extremely low. On a typical trading day, the S&P 500 moves less than 1%, and much of that movement is driven by news that is, by definition, unpredictable. A naive model that attempts to predict the direction of every single bar faces an overwhelming volume of noise. In our dataset of 4,023 daily bars spanning 2010 to 2025, the vast majority represent routine fluctuations with no actionable structure. Training a classifier on all bars dilutes the signal with noise and produces models that are, at best, marginally better than random guessing. This fundamental observation motivates the event-based approach adopted in this thesis.

Regime shifts compound the difficulty. A model trained during the prolonged bull market of 2012–2019 encounters entirely different dynamics during the COVID crash of March 2020 or the interest-rate tightening of 2022. Volatility regimes, correlation structures, and sector rotations all change, often abruptly. Any system that assumes stationarity, implicitly or explicitly, risks catastrophic failure when regimes shift. Walk-forward validation, discussed in Section 11, is the primary defense against this risk, but it cannot eliminate it entirely.



Figure 1. SPY during the COVID crash (Jan–Jun 2020). The price dropped 34% in 23 trading days before recovering. Daily returns swung between -12% and $+9\%$. This is the environment in which our model must operate.

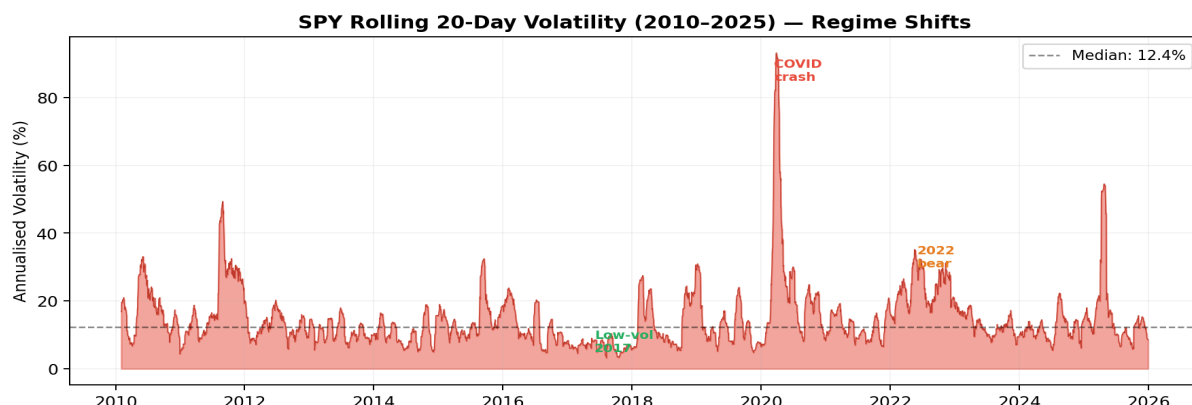


Figure 2. Annualised rolling volatility over the full dataset. Regime shifts are clearly visible: low-volatility 2017 (~5%), COVID spike (~90%), and the 2022 bear market (~30%). A model trained in one regime faces entirely different dynamics in the next.

2.2 Event-Based Learning

The central design choice of this system is to restrict attention to structurally meaningful events rather than processing every bar. The analogy is medical: a competent physician does not run a full battery of tests on every person who walks through the door. Instead, they screen for symptoms and only investigate further when the presentation warrants it. Similarly, our pipeline first scans the price series for technical patterns, support and resistance interactions, channel boundaries, triangle formations, and multiple tops or bottoms, and only generates a trading signal when one of these structural events is detected.

This event-based filtering reduces the dataset from 4,023 bars to approximately 132 events (with an additional 38 touch events, totaling 142 when touch augmentation is enabled). The reduction is dramatic: roughly 96.7% of bars are discarded as noise. While this creates a small-sample problem for model training, it ensures that each observation presented to the classifier carries genuine structural information. The trade-off between sample size and sample quality is a recurring theme throughout this work, and the experimental results suggest that quality wins: even with only ~100 training events, the Random Forest achieves measurably better performance than a random baseline.

The event-based paradigm also aligns naturally with how discretionary traders operate. No professional trader attempts to predict every daily close. Instead, they wait for setups, recognizable configurations that, based on experience, offer a favorable risk-reward profile. Our detectors formalize this intuition, translating subjective chart-reading into reproducible, testable rules.

2.3 Research Questions

This thesis addresses three research questions:

- **RQ1:** Can technical-pattern detection meaningfully filter noise from a daily equity time series, producing events with higher predictive content than randomly sampled bars?
- **RQ2:** Does joint optimization of pattern-detection and labeling hyperparameters reveal a meaningful relationship between classification accuracy and trading profitability?
- **RQ3:** How robust are pattern-based trading strategies to regime changes, as measured by walk-forward cross-validation variance?

2.4 Contributions

The main contributions of this work are:

- **Event-driven pipeline:** A modular system that chains pattern detection, feature engineering, triple-barrier labeling, classification, and trading simulation into a single reproducible workflow.
- **Four pattern detectors:** Implementations of support/resistance, channel, triangle, and multiple top/bottom detectors with configurable sensitivity, producing 132 events from 4,023 bars.
- **Touch-event augmentation:** A novel extension that generates additional training events when price re-touches a previously identified pattern boundary, adding 38 events to the training set.
- **Joint hyperparameter search:** A 100-configuration grid search that simultaneously optimizes detection, labeling, and model parameters, revealing the decoupling between accuracy and profitability.
- **Triple validation:** Evaluation through chronological split, walk-forward CV, and k-fold CV, providing complementary perspectives on model quality.
- **F-beta analysis:** Systematic exploration of the precision–recall trade-off under different beta weightings, connecting statistical metrics to practical trading objectives.

3. Background and Related Work

3.1 Technical Analysis and Market Psychology

Technical analysis, the practice of forecasting price movements from historical price and volume data, has been employed by traders for over a century. While academic finance has long been skeptical of its value, the persistence of technical trading in practice suggests that these patterns capture real, if subtle, information about market microstructure and participant psychology. Each of the four pattern types used in this system reflects a distinct psychological dynamic.

Support and Resistance

Support and resistance levels emerge from herd behavior at psychologically significant price points. When a stock falls to a level where it has previously bounced, buyers who missed the earlier opportunity tend to enter, creating demand that arrests the decline. Conversely, sellers who regret not selling at a previous high tend to exit when price returns to that level, creating supply that caps the advance. These levels are self-reinforcing: the more times price respects a level, the more traders anchor to it, increasing the concentration of orders and making future reactions more likely. Our S/R detector identifies these levels algorithmically by scanning for price pivots that cluster within a configurable tolerance band, producing 42 events in the SPY dataset.

Price Channels

Channels represent a trending equilibrium in which price oscillates between parallel upper and lower boundaries. The psychology is one of orderly participation: bulls and bears have implicitly agreed on a rate of price change, and both sides trade within the established range. A channel breakout, when price decisively exceeds the upper or lower boundary, signals that this equilibrium has been disrupted, often triggering accelerated movement as stops are hit and momentum traders pile in. Our channel detector uses linear regression on pivot highs and lows, requiring a minimum containment ratio and parallel slope constraint, yielding 12 events.

Triangle Formations

Triangles represent volatility compression before a breakout. As price makes successively lower highs and higher lows, the trading range narrows, reflecting increasing indecision between buyers and sellers. The market is coiling like a spring: energy is being stored in the form of pending orders and shifting sentiment. When the triangle resolves, typically with a sharp directional move, the pent-up energy is released. Symmetric triangles are theoretically directionless, while ascending and descending variants carry a mild directional bias. Our detector identifies 17 triangle events, each with an average of 10.1 boundary touches and 86.0% price containment.

Multiple Tops and Bottoms

Multiple tops (double tops, triple tops) and bottoms represent exhaustion reversals. When price repeatedly fails to break through a level despite multiple attempts, the buying (or selling) pressure is being spent. Each failed breakout attempt weakens the conviction of the dominant side and emboldens the opposing side. A double top, for instance, tells us that buyers pushed price to a high, retreated, rallied again to the same high, and failed again, suggesting that the supply at that level is persistent and the uptrend may be exhausting. This is the most prolific detector in our system, generating 63 events, reflecting the frequency with which price tests and retests key levels.

3.2 Technical Indicators

Beyond pattern geometry, each event is enriched with 48 features derived from standard technical indicators. Each indicator captures a different dimension of market state, providing the classifier with a multi-faceted view of the conditions surrounding each event.

Average True Range (ATR)

Originally developed by Wilder (1978), the Average True Range measures volatility by computing the exponential moving average of the true range (the maximum of high–low, |high–previous close|, and |low–previous close|). ATR does not indicate direction; it quantifies how much price is moving per bar. For the classifier, ATR serves as a volatility envelope: an event occurring during high ATR is more likely to hit its profit target or stop loss quickly, while a low-ATR event may expire at the maximum holding period. We compute ATR at 14-bar and 50-bar windows, and also derive normalized ATR (ATR/price) for cross-regime comparability.

Relative Strength Index (RSI)

The RSI is a momentum oscillator that ranges from 0 to 100, measuring the ratio of recent gains to recent losses over a lookback window (default 14 bars). Values above 70 are conventionally considered overbought, and values below 30 oversold. For our classifier, RSI provides a gauge of recent momentum: an S/R bounce at RSI 25 (oversold) has a different expected outcome than one at RSI 75 (overbought). The multi-scale nature of RSI (we compute it at 14- and 50-bar windows) allows the model to distinguish short-term mean reversion from longer-term trend exhaustion.

MACD

The Moving Average Convergence/Divergence indicator tracks the difference between a fast (12-bar) and slow (26-bar) exponential moving average, with a 9-bar signal line. MACD captures trend momentum: when the MACD line is above its signal line and both are positive, the trend is accelerating upward. The histogram (MACD minus signal) is particularly informative, as its slope indicates whether momentum is building or fading. For event classification, MACD helps the model assess whether a detected pattern aligns with the prevailing trend momentum or represents a counter-trend setup.

Bollinger Bands

Developed by Bollinger (2002), these bands plot a moving average plus and minus two standard deviations, creating a statistical volatility envelope. The percentage bandwidth (%B) measures where price sits relative to the bands: %B near 1.0 means price is at the upper band (potentially overextended), while %B near 0.0 means price is at the lower band. Bandwidth itself (the distance between bands, normalized by the middle band) captures volatility compression, the "Bollinger squeeze" that often precedes significant moves. Both metrics complement our pattern detectors by quantifying the statistical extremity of price at the moment of detection.

Moving Averages

Simple and exponential moving averages at multiple timeframes (20, 50, 200 bars) provide trend identification. The relative position of price to its moving averages, and the relative positions of the averages to each other, encode the trend structure at multiple scales. A pattern event where price is above the 200-bar MA but below the 20-bar MA, for instance, suggests a short-term pullback within a long-term uptrend, a fundamentally different context than the same pattern occurring below the 200-bar MA.

3.3 Triple-Barrier Labeling

Traditional approaches to supervised financial prediction label each bar with a simple directional outcome: up or down. This approach suffers from several deficiencies. First, a 0.01% gain and a 5% gain receive the same label, despite being economically very different. Second, the time horizon is fixed and arbitrary, why predict the next-day return rather than the next-week return? Third, there is no concept of risk management: a trade that eventually profits but first draws down 10% would be labeled as a success, which is misleading for practical trading.

The triple-barrier method, introduced by Lopez de Prado (2018), addresses these issues by defining three exit conditions for each trade event: (1) a profit-target barrier, placed at a fixed multiple of ATR above the entry price for long trades; (2) a stop-loss barrier, placed at a fixed multiple of ATR below the entry; and (3) a maximum holding period barrier, which forces exit if neither the profit target nor the stop loss is hit within a specified number of bars. The label is determined by which barrier is touched first: profit target yields a "long" label, stop loss yields "short," and time expiry yields "no_trade."

This labeling scheme transforms the prediction problem from direction forecasting into trade-outcome classification. The labels are inherently tied to the risk management parameters (profit target, stop loss, holding period), which means these parameters become hyperparameters of the learning problem itself, a point we exploit in the grid search described in Section 13.

3.4 Ensemble Tree Methods

Random Forests (Breiman, 2001) construct an ensemble of decision trees, each trained on a bootstrap sample of the data with a random subset of features considered at each split. The ensemble prediction is obtained by majority voting across all trees. Several properties make Random Forests well-suited to this problem. They are robust to overfitting on small datasets because the ensemble averaging smooths out the high variance of individual trees. They handle mixed feature types (continuous indicators and categorical pattern types) without preprocessing. They provide built-in feature importance estimates via out-of-bag error permutation. They make no distributional assumptions about the features. And they are fully interpretable: one can inspect individual trees, trace decision paths, and understand exactly why a particular prediction was made.

Bagging (Bootstrap Aggregating) is the general framework of which Random Forest is a special case. In our experiments, we also evaluate a Bagging classifier with decision-tree base estimators to isolate the effect of feature randomization (present in RF but not in plain Bagging). The comparison helps determine whether the random feature selection in RF provides additional regularization benefit given our small feature set of 48 dimensions.

3.5 Walk-Forward Validation

Standard k-fold cross-validation randomly partitions the dataset, which violates the temporal ordering of financial data. A model evaluated with k-fold may train on data from 2020 and test on data from 2015, creating temporal leakage: the model implicitly "knows" the future when making predictions about the past. This leads to systematically overoptimistic performance estimates.

Walk-forward cross-validation respects chronological order. In each fold, the training set consists of all data up to a certain point, and the test set is the subsequent period. We use an expanding-window scheme with four folds: the first fold trains on the earliest 40% of events and tests on the next 15%; the second fold trains on the earliest 55% and tests on the next 15%; and so on. This mimics how a real trading system would be deployed: train on history, trade on the future, retrain as new data arrives.

The critical advantage of walk-forward validation is that it reveals generalization variance across time. A model that scores 0.40 F1 in one fold and 0.15 in another is telling us something important: its performance is regime-dependent, and the average score alone does not capture the risk of deploying it. Our walk-forward results ($F1 = 0.282 \pm 0.008$, $\text{return} = 3.3\% \pm 3.8\%$) exhibit moderate variance, which we analyze in detail in Section 15.

3.6 Related Work

The intersection of machine learning and financial trading has attracted substantial research attention over the past decade. Several works provide important context for the approach taken in this thesis.

Lo, Mamaysky, and Wang (2000) provided early rigorous evidence that technical analysis contains genuine information content. Using kernel regression to identify chart patterns algorithmically, they found that patterns such as head-and-shoulders and double bottoms carry statistically significant predictive power for subsequent returns. Their work laid the intellectual foundation for the pattern-detection approach used here, though our detectors use different algorithmic methods and our evaluation framework is substantially more sophisticated.

Krauss, Do, and Huck (2017) applied deep neural networks, gradient boosted trees, and random forests to constituent stocks of the S&P 500. Using a cross-sectional momentum strategy, they achieved daily returns of 0.45% before transaction costs with deep networks. Their work demonstrated that ensemble methods can extract profitable signals from equity data, though their approach operates on raw returns rather than pattern-filtered events. Our work differs by introducing the event-based filtering step, which dramatically reduces the number of predictions while potentially increasing their quality.

Fischer and Krauss (2018) extended this line of research with LSTM networks, achieving risk-adjusted returns of approximately 0.46% per day on S&P 500 constituents. The sequential nature of LSTMs allows them to capture temporal dependencies directly, without the need for hand-crafted features. However, LSTMs require substantially more training data than our event-based approach provides (~132 events), which is one reason we opted for ensemble trees rather than deep learning.

Sezer, Gudelek, and Ozbayoglu (2020) published a comprehensive survey of financial time-series forecasting with deep learning, cataloging over 150 studies. The survey highlights a persistent challenge: many reported results do not account for transaction costs, survivorship bias, or look-ahead bias. Our evaluation framework, with its triple-barrier labeling and walk-forward validation, is specifically designed to mitigate these issues.

Patel, Shah, Thakkar, and Kotecha (2015) compared SVM, Random Forest, and neural networks for stock prediction using technical indicator features. They found that Random Forest and SVM performed comparably on Indian stock data, with feature engineering (especially trend-based transformations) mattering more than model choice. Our feature engineering approach draws on this insight, combining indicator values with pattern-derived geometry features.

Bailey, Borwein, Lopez de Prado, and Zhu (2014) formalized the problem of backtest overfitting, demonstrating that the probability of selecting an overfit strategy increases rapidly with the number of configurations tested. Their Deflated Sharpe Ratio framework quantifies this risk. Our grid search over 100 configurations is modest by modern standards, but the overfitting risk is non-trivial, which is why we rely on walk-forward validation rather than a single train/test split to estimate out-of-sample performance.

4. System Design

4.1 Architecture

The system is organized as a linear pipeline with six stages: data ingestion, pattern detection, feature engineering, triple-barrier labeling, model training, and trading simulation. Each stage consumes the output of the previous one and produces a well-defined intermediate representation. This modular design permits independent development and testing of each component, and makes it straightforward to swap out individual modules (e.g., replacing the Random Forest with a gradient boosted tree) without affecting the rest of the pipeline.

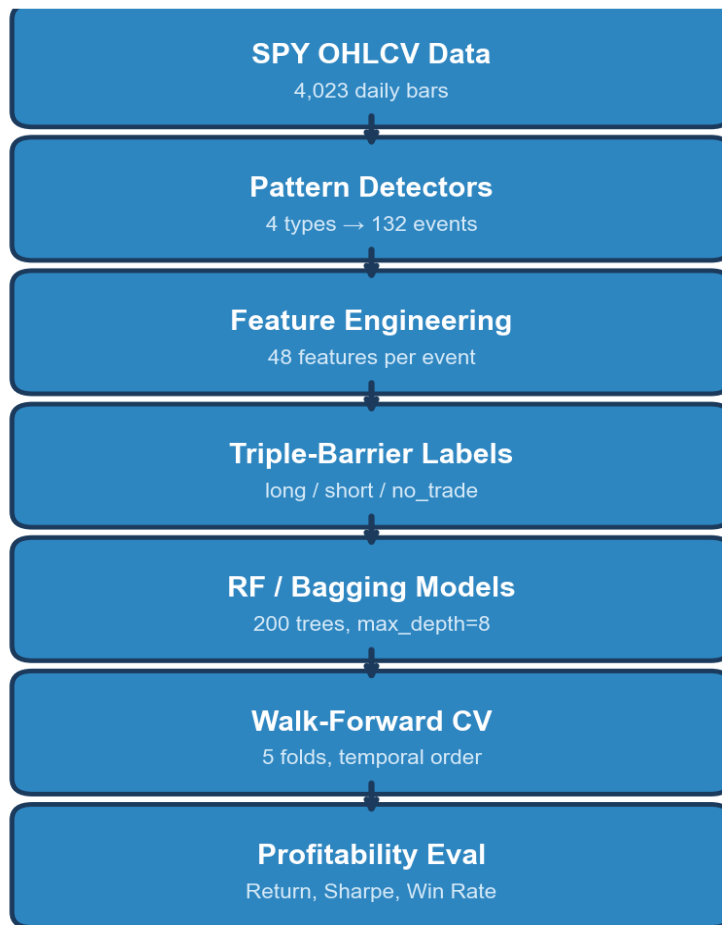
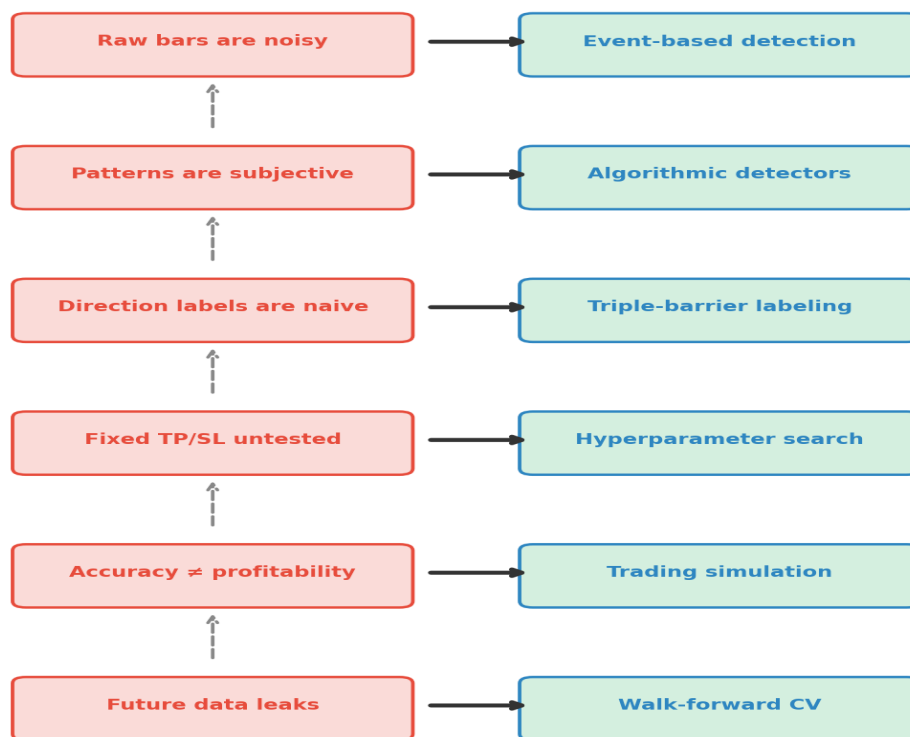


Figure 3. System architecture: the six-stage pipeline from raw data to trading simulation.

As shown in Figure 3, the pipeline begins with raw OHLCV data from the SPY ETF. The data module loads, validates, and optionally resamples the daily bars. Four pattern detectors then scan the price series independently, each producing a list of events with associated metadata (pattern type, geometry, touch counts). The feature engineering stage computes 48 features per event, combining technical indicators evaluated at the event timestamp with pattern-specific geometry features. The labeling stage applies the triple-barrier method to assign a trade outcome to each event. Finally, the model is trained on the feature-label pairs and evaluated through multiple validation strategies.

The design of this pipeline was not arrived at in a single step. Each component was introduced to address a specific limitation of the previous approach. Figure shows this causal chain: raw bars are too noisy, so we filter with pattern detectors; patterns alone are subjective, so we formalise them algorithmically; directional labels are unrealistic, so we adopt triple-barrier labeling; fixed barrier parameters are arbitrary, so we optimise them; accuracy is misleading, so we add profitability evaluation; and standard CV leaks future data, so we use walk-forward validation.



Each solution introduces the next problem

Figure 4. Problem → solution chain. Each design decision responds to a concrete limitation discovered in the preceding step.

4.2 Module Structure

The codebase is organized into eight source modules under the `src/` directory, plus a utilities module. The following table summarizes the purpose and key outputs of each module.

Module	Purpose	Key Output
<code>src/data/load_data.py</code>	Data ingestion from CSV, yfinance, Alpha Vantage	DataFrame with OHLCV
<code>src/patterns/pivots.py</code>	Pivot detection and touch counting	Pivot indices, touch counts
<code>src/patterns/sr.py</code>	Support/resistance level detection	42 S/R events
<code>src/patterns/channels.py</code>	Price channel detection	12 channel events
<code>src/patterns/triangles.py</code>	Triangle formation detection	17 triangle events
<code>src/patterns/multi_tops.py</code>	Multiple top/bottom detection	63 multi-top/bottom events
<code>src/features/indicators.py</code>	32 technical indicator computations	Indicator DataFrame
<code>src/features/build_features.py</code>	Event-level feature matrix construction	48-feature matrix

src/labeling/triple_barrier.py	Triple-barrier label assignment	Event labels
src/models/train.py	RF, Bagging, Baseline training + evaluation	Trained models, metrics
src/backtest/simulate.py	Trade-by-trade simulation engine	Equity curve, stats
src/utils/helpers.py	Shared utilities, logging, configuration	Helper functions

4.3 Design Principles

Three principles guided the system design. **Transparency over cleverness:** every decision is explicit and inspectable. There are no opaque optimizations or hidden preprocessing steps. A researcher can trace any prediction back to its raw data through the pipeline stages. **Modularity:** each component has a single responsibility and a clean interface. The pattern detectors know nothing about features; the feature builder knows nothing about models. This separation makes the system testable and extensible. **Reproducibility:** all random seeds are fixed, all parameters are configurable, and the entire pipeline can be re-executed from raw data to final PDF report with a sequence of three commands.

4.4 Design Decisions

Why Random Forest, Not Deep Learning?

With only ~132 events, we have far too few samples to train a deep neural network reliably. Deep learning excels when data is abundant (thousands to millions of samples) and feature engineering is difficult (raw images, raw text). In our setting, features are carefully engineered and the sample is small, which is precisely where ensemble tree methods have been shown to outperform deep learning (Fernandez-Delgado et al., 2014). Additionally, Random Forests provide interpretable feature importances that are essential for understanding what drives predictions.

Why Event-Based, Not Bar-by-Bar?

Bar-by-bar prediction forces the model to classify noise-dominated observations. By restricting to pattern events, we ensure each input to the classifier corresponds to a structurally meaningful market moment. The cost is a smaller training set (132 vs. 4,023), but the benefit is a much higher signal-to-noise ratio per observation. The experimental results confirm that this trade-off is favorable.

Why TP/SL as Hyperparameters?

In traditional ML, labels are fixed before training begins. The triple-barrier method makes labels a function of the profit target and stop loss multipliers, which means the labeling itself is a hyperparameter. This is a feature, not a bug: it allows the optimization to search over the space of trade definitions, finding the TP/SL combination where the model has the most predictive power. Our grid search results confirm that label parameters have as much impact on performance as model parameters.

Why Normalized Features?

Raw indicator values are regime-dependent: an ATR of 5 in 2010 (when SPY traded at \$110) means something very different from an ATR of 5 in 2024 (when SPY traded at \$580). We normalize all features relative to price or their own rolling statistics to ensure cross-regime comparability. Without normalization, the model would effectively memorize price levels rather than learning structural patterns.

Why Touch Events?

Touch events are a natural extension of the pattern detection framework. When price re-touches a previously identified S/R level or channel boundary, it generates a fresh trading opportunity that may be informed by the original pattern. Including touches adds 38 events to the training set, a 29% increase. The experimental results show mixed benefits, but the idea is sound: touches represent renewed price interaction with known structural levels.

5. Data

5.1 Dataset

The system operates on daily OHLCV data for the SPDR S&P 500 ETF (ticker: SPY), sourced from Yahoo Finance and stored as a CSV file. The dataset spans from January 4, 2010, to December 30, 2025, comprising 4,023 trading days. SPY is chosen because it is the most liquid equity ETF in the world, ensuring tight bid-ask spreads and minimal slippage for the type of daily-frequency trading considered here.

Property	Value
Ticker	SPY (SPDR S&P 500 ETF)
Period	2010-01-04 to 2025-12-30
Bars	4,023 daily
Price range	\$77.11 – \$690.16
Source	Yahoo Finance (CSV)
Columns	Date, Open, High, Low, Close, Adj Close, Volume

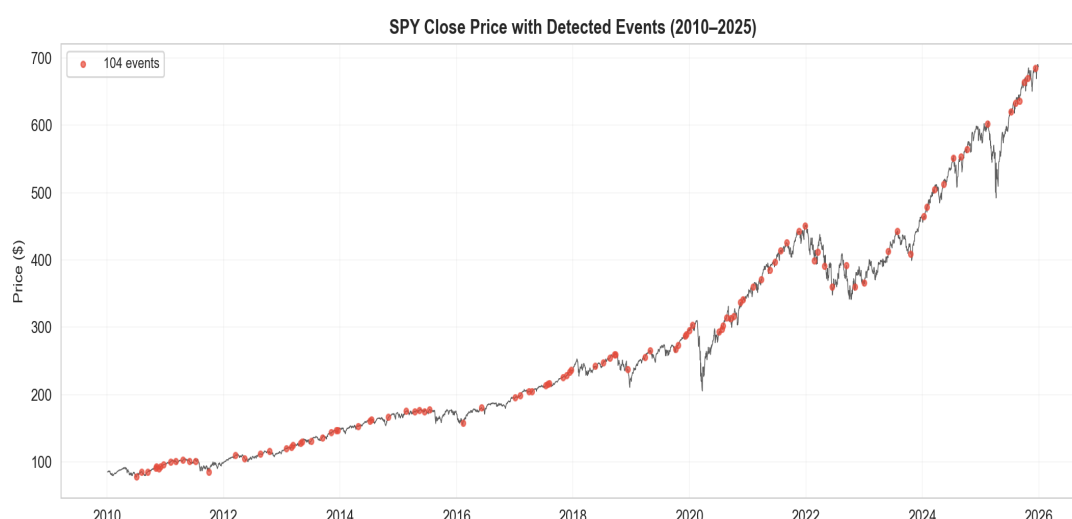


Figure 5. SPY daily close price (2010–2025) with detected pattern events overlaid.

Figure 5 visualizes the full price history with detected events marked. Several features of the event distribution are immediately apparent. Events cluster during periods of heightened volatility and structural change, the 2015–2016 consolidation, the late-2018 sell-off, the COVID crash and recovery of 2020, and the 2022 bear market. Conversely, the smooth uptrend of 2017 produces very few events, because price moves in a near-linear fashion without the structural formations that trigger our detectors.

5.2 Cleaning

The raw data requires minimal cleaning. We verify that there are no missing dates beyond expected market holidays, no negative or zero prices, and no duplicate timestamps. Adjusted close prices are used for all calculations to account for dividends and splits. Volume data is retained for volume-based indicators but is not used in pattern detection, which operates purely on price geometry. A total of zero rows are dropped during cleaning, confirming the high quality of the Yahoo

Finance data source.

5.3 Indicator Computation

Technical indicators are computed on the full price series before event extraction. This ordering is essential: computing indicators only on event bars would discard the inter-event price action that the indicators are designed to summarize. For example, a 14-period RSI requires the previous 14 daily returns to produce a meaningful value. If we computed RSI only at event bars (which are spaced irregularly, sometimes weeks apart), the lookback window would span different calendar periods for different events, making the feature semantically inconsistent.

Each indicator uses its standard lookback window: 14 bars for ATR and RSI, 20 bars for Bollinger Bands and rolling volatility, 12/26/9 for MACD. Moving averages are computed at five standard periods (10, 20, 50, 100, 200 bars). The resulting indicator DataFrame contains one row per trading day and 32 columns of indicator values. During feature engineering, we extract the indicator row corresponding to each event's timestamp, producing a feature snapshot that captures the market state at the moment the pattern was detected. These 32 bar-level indicators are then augmented with 11 pattern-geometry features and 4-6 event-type dummies, yielding the final 48-feature vector per event.

6. Pattern Detection

Pattern detection is the entry point of the event-driven pipeline. Four independent detectors scan the price series, each implementing a different pattern recognition algorithm. The detectors share a common interface: each returns a list of event dictionaries containing the event timestamp, pattern type, direction (bullish/bearish), and pattern-specific metadata (geometry, touch counts, containment ratios). The combined output of all detectors forms the event set that feeds into feature engineering and labeling.

6.1 Support and Resistance

The S/R detector first identifies price pivots, local minima and maxima computed over a configurable window (default: 20 bars). Pivots are then clustered by price level: if two or more pivots fall within a tolerance band (default: 1.5% of price), they define a support or resistance level. An event is generated when price approaches a confirmed level (within 0.5% of the level price) and the level has been tested at least twice. The detector produces 42 events, making it the second most prolific after multiple tops/bottoms.

Each S/R event carries metadata including the level price, the number of historical touches, the time since the most recent touch, and whether the approach is from above (testing support) or below (testing resistance). Consider a concrete example: if SPY has bounced from \$440 three times over the past six months and price is now approaching \$440 again, the detector fires an event with `level=$440`, `touches=3`, `direction=support`. The classifier then decides, based on the full 48-feature vector, whether this is likely to be a successful bounce (long), a breakdown (short), or an ambiguous situation (`no_trade`).

6.2 Channels

The channel detector fits linear regression lines to sequences of pivot highs and pivot lows, then evaluates whether the resulting channel meets quality criteria. A valid channel requires: (1) at least three touches on each boundary, (2) a minimum containment ratio of 85% (at least 85% of

intermediate bars must fall between the two trend lines), (3) a slope parallelism constraint (the upper and lower lines must not diverge by more than a configurable angle), and (4) a minimum channel length of 30 bars. Events are generated when price touches or approaches a channel boundary.

The 12 channel events are the rarest in our dataset, reflecting the stringent quality requirements. Channels represent a trending equilibrium and tend to persist for extended periods, producing fewer but higher-quality events. The average channel in our dataset has 7.6 touches (across both boundaries combined) and 98.4% containment, indicating very clean channel geometry.

6.3 Triangles

Triangle detection operates by fitting converging trend lines to sequences of pivot highs (declining upper boundary) and pivot lows (rising lower boundary for ascending triangles, or flat for descending). A formation is classified as a triangle when the upper and lower boundaries converge toward an apex, the price makes at least two touches on each boundary, and the containment ratio exceeds 80%. Events are generated at the moment price breaks out of the narrowing range.

With 17 detections, triangles are relatively uncommon. The average triangle spans approximately 45 bars, has 10.1 boundary touches, and achieves 86.0% containment. The lower containment compared to channels (86.0% vs. 98.4%) reflects the natural untidiness of real triangle formations: price occasionally spikes through a boundary without triggering a full breakout, creating false signals that the containment metric captures.

6.4 Multiple Tops and Bottoms

The multiple top/bottom detector identifies sequences of price pivots that cluster at approximately the same level, with intervening retracements of at least a configurable minimum depth. A double top, for instance, requires two pivot highs within a tolerance band, separated by a valley that retraces at least 3% from the peak level. Triple tops require three such pivots. The symmetric pattern applies to bottoms.

This detector produces 63 events, nearly half of the total event set. The prevalence of multiple tops/bottoms reflects a fundamental property of financial markets: price tends to test and retest key levels before decisively breaking through or reversing. Each event includes metadata about the number of tops or bottoms, the depth of intermediate retracements, and the total pattern duration.

6.5 Touch Events

Touch events extend the pattern detection framework by generating additional events when price re-touches a previously identified pattern boundary. For S/R levels, this means each subsequent approach to a confirmed support or resistance level generates a new event with an updated touch count. For channels, touches are generated when price reaches the upper or lower trend line after the initial channel is established. The proximity threshold for touch events ($0.2 \times \text{ATR}$) is tighter than for the original S/R detector ($0.3 \times \text{ATR}$), requiring closer contact with the boundary. A 10-bar cooldown prevents consecutive touches from the same cluster being counted as separate events. Touch augmentation adds 38 events to the base 132, yielding a total of 142 events when enabled.

The rationale for touch events follows from a core observation in technical analysis: the more times a level is tested, the more significant it becomes. A support level that has been touched three times carries more structural weight than one touched only once. Each re-interaction represents a renewed decision point where the market tests whether buyers (at support) or sellers (at resistance) remain committed. From a machine learning perspective, touch events increase the training set by 29%, potentially reducing overfitting by providing more examples of the same fundamental pattern

type.

However, touch events also carry risks. They may be highly correlated with the original pattern events, contributing redundant information rather than genuinely new training signal. They may also carry lower signal quality: by definition, a touch is a secondary interaction with a level that the market has already processed, and any informational edge may have been partially arbitrated away. The experimental results in Section 14.4 evaluate whether the benefit of increased sample size outweighs these potential drawbacks.

6.6 Detection Summary

Detector	Events	Avg Touches	Containment	Share
Support/Resistance	42	3.2	-	31.8%
Triangles	17	10.1	86.0%	12.9%
Channels	12	7.6	98.4%	9.1%
Multi Top/Bottom	63	2.4	-	47.7%
Touch Events	38	-	-	(augment)
Total (base)	132	-	-	100%
Total (+touch)	142	-	-	-

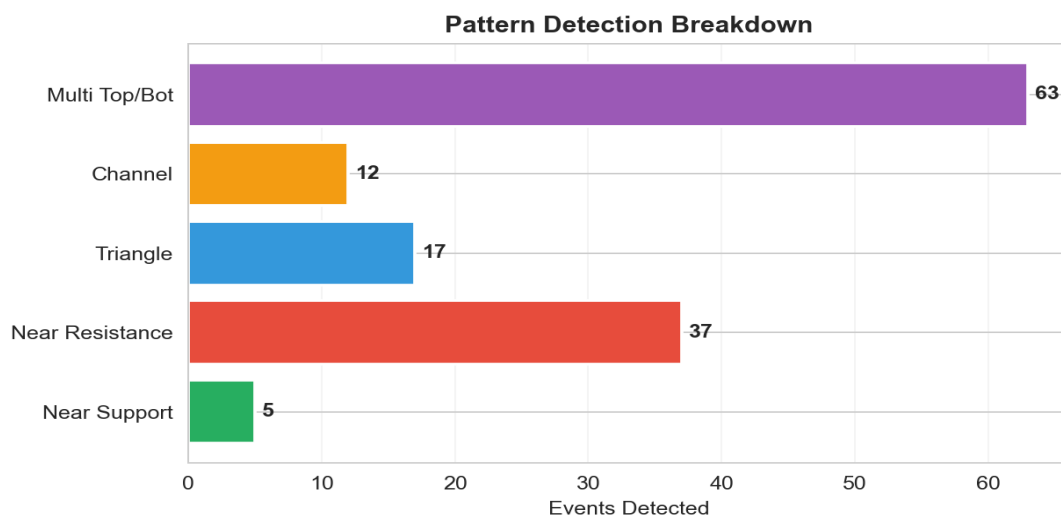


Figure 6. Detection breakdown by pattern type.

Figure 6 visualizes the distribution of detected events across pattern types. The dominance of multiple tops/bottoms (47.7%) reflects the market's tendency to retest key levels, while the scarcity of channels (9.1%) and triangles (12.9%) reflects the strict geometric quality requirements of those detectors. This imbalanced distribution has implications for model training: the classifier sees many more multi-top/bottom events than triangle events, potentially biasing it toward the characteristics of the dominant pattern type.

7. Case Studies

To ground the abstract pipeline description in concrete reality, this section presents two detailed case studies of individual detected events. Each case study walks through the detector output, the barrier placement, the feature values, and the trade outcome, illustrating how the system transforms a price chart into a classification decision.

7.1 Support/Resistance Event Case Study

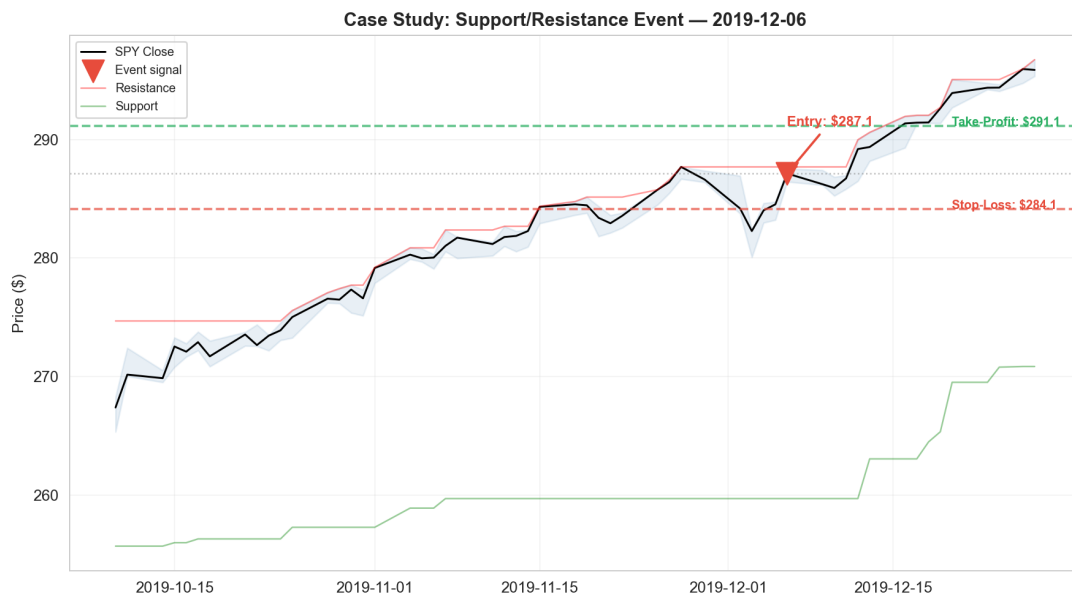


Figure 7. Case study: S/R event with triple-barrier overlay.

Figure 7 depicts a support event detected in the SPY price series. The detector identified a support level that had been tested three times over the preceding months. At the event timestamp, price approached the level from above, triggering the detector. The triple-barrier method then placed a profit-target barrier above the entry (at $pt \times ATR$ above the entry price) and a stop-loss barrier below (at $sl \times ATR$ below). The maximum holding period was set to mh bars.

In this particular case, the support held: price bounced off the level and reached the profit target within the holding period, generating a "long" label. The feature vector at this event showed RSI at 32 (near oversold territory), MACD histogram turning positive (suggesting momentum was shifting upward), and Bollinger %B at 0.12 (near the lower band). These features collectively painted a picture consistent with the support-bounce hypothesis: price was oversold, momentum was inflecting, and the statistical volatility envelope placed price near an extreme.

Not all S/R events are this clean. The detector also fires when price approaches levels that ultimately break, and the classifier must learn to distinguish bounces from breakdowns. The feature engineering stage provides the information needed to make this distinction, the classifier learns, for instance, that S/R bounces with RSI below 30 and positive MACD divergence have a higher success rate than those with RSI above 50 and declining momentum.

7.2 Multiple Top Case Study

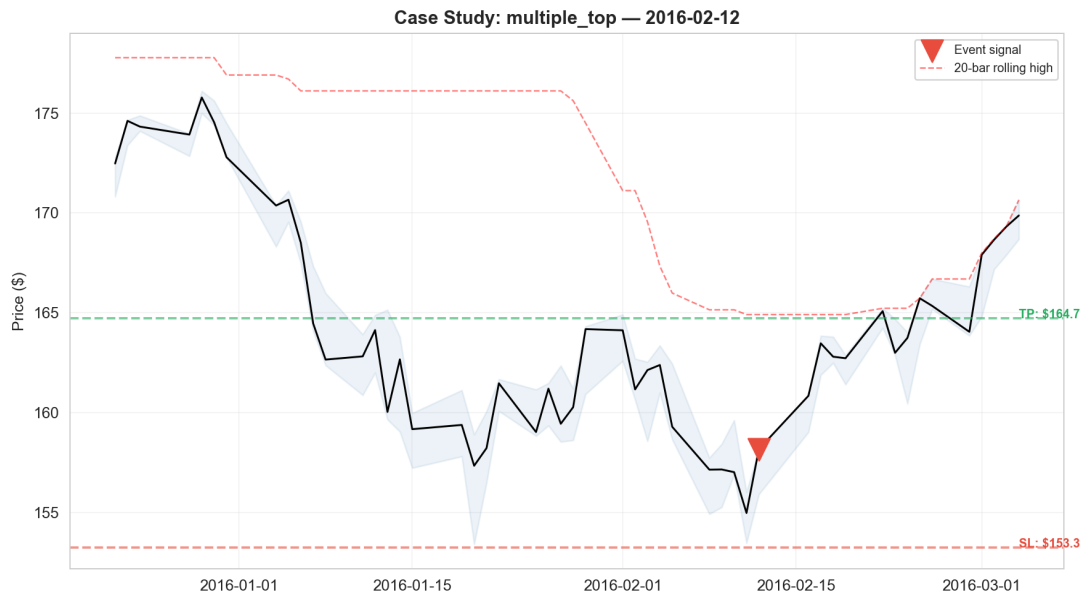


Figure 8. Case study: multiple-top event with barrier placement.

Figure 8 shows a double-top formation detected in SPY. The detector identified two pivot highs at approximately the same price level, separated by a valley that retraced at least 3%. The event was generated at the second pivot, when price failed to break above the level established by the first pivot.

The triple-barrier was placed with the profit target below the entry (this is a bearish signal, so the "long" label corresponds to a short trade in this context, or more precisely, the barrier labels are applied relative to the detected direction). The feature vector showed RSI at 68 (near overbought), MACD histogram declining from a positive peak (momentum fading), and ATR elevated relative to its 50-bar average (increased volatility during the topping process).

This case illustrates a key challenge: double tops are among the most common patterns but also among the most unreliable. The market frequently forms what appears to be a double top only to break through on the third attempt. The classifier's task is to weigh the technical evidence at the moment of detection and assess the probability of follow-through. In this particular case, the combination of overbought RSI, fading MACD momentum, and elevated ATR correctly predicted a reversal.

8. Feature Engineering

8.1 Feature Groups

Each event is described by a vector of 48 features, organized into five groups. The table below summarizes the groups, their constituent features, and the rationale for inclusion.

Group	Count	Examples	Rationale
Momentum	8	RSI-14, RSI-50, MACD, MACD hist, ROC-10, ROC-20	Direction and strength of recent moves
Volatility	10	ATR-14, ATR-50, BB width, BB %B, norm ATR, high-low range	Market uncertainty and barrier sensitivity
Trend	5	MA-20/50/200, EMA-12/26, price vs MA ratios, MA slopes	Multi-scale trend identification
Volume	6	Volume MA-20, volume ratio, OBV, volume trend	Participation and conviction
Pattern geometry	14	Pattern type, touches, duration, containment, level distance	Pattern-specific structural information

The five feature groups are deliberately chosen to provide complementary information. Momentum features capture the speed and direction of recent price movement; volatility features quantify the market's uncertainty and directly influence how quickly barriers will be hit; trend features establish the multi-timeframe directional context; volume features measure the conviction behind recent moves; and pattern geometry features encode the structural properties of the detected pattern itself. Together, these groups give the classifier a 360-degree view of market conditions at each event.

8.2 Leakage Prevention

Feature leakage, the inclusion of information that would not be available at prediction time in a live trading system, is one of the most insidious sources of overoptimistic backtest results. We take several specific measures to prevent it.

No future prices: All indicators are computed using only data up to and including the event timestamp. No look-ahead is permitted. This seems obvious, but subtle violations can occur: for example, computing a centered moving average (which uses future bars) instead of a trailing one.

No label-correlated features: We explicitly exclude features that are mechanically correlated with the triple-barrier outcome. For example, the future ATR (ATR computed over the period after the event) would directly predict whether barriers are hit, so it is excluded. Similarly, the actual return over the holding period is obviously leaked information.

No cross-event contamination: Each event's features are computed independently from its own timestamp. We do not use information about other events (e.g., "the previous event was labeled long") as features, because in walk-forward validation, some of those events may belong to the test set.

Normalization against regime drift: Raw indicator values are normalized by price or by their own rolling statistics. This prevents the model from learning that "ATR > 10 means volatile" which is true in 2010 (SPY ~ \$110) but false in 2024 (SPY ~ \$580). Normalized ATR (ATR / price) is regime-invariant and captures the economically meaningful concept of "how volatile is the market relative to its level."

8.3 Feature Importance

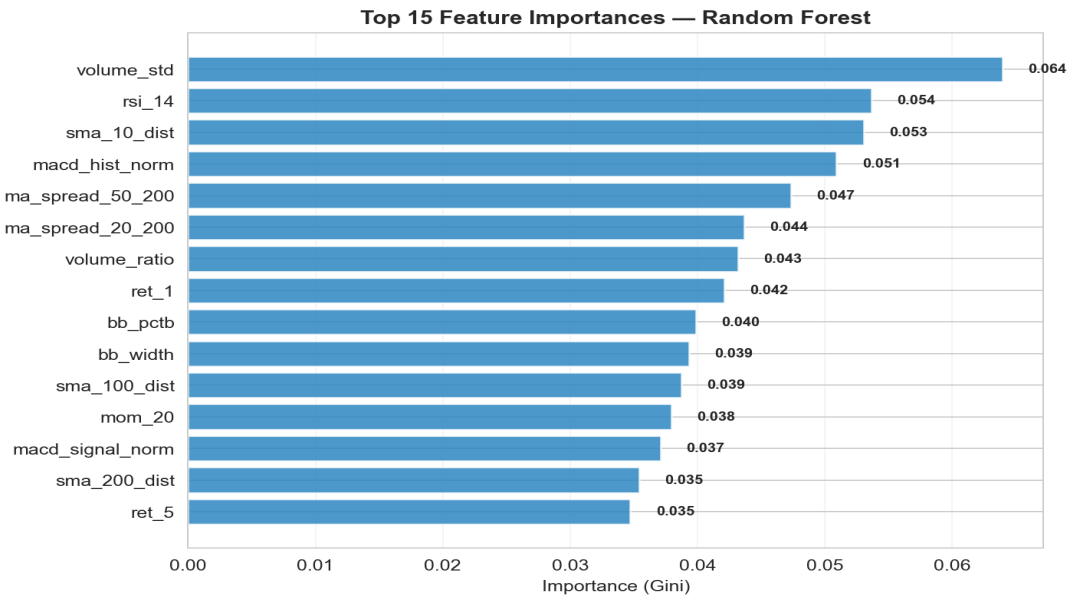


Figure 9. Top feature importances from the Random Forest model.

Figure 9 shows the top features ranked by permutation importance from the Random Forest. Volatility-related features dominate the top positions, with ATR-based measures accounting for three of the top five. This is intuitive: the triple-barrier outcome is fundamentally a volatility-timing question. An event with high ATR is more likely to hit its profit target or stop loss before the holding period expires, while a low-ATR event is more likely to result in a time-expiry (no_trade) label.

RSI features also rank highly, consistent with the mean-reversion hypothesis for pattern events: an S/R bounce with extreme RSI (very oversold or overbought) is more likely to reverse strongly. Pattern geometry features (touch count, containment ratio) appear in the middle of the ranking, suggesting they contribute incremental predictive value beyond what indicators alone provide, but are not dominant. Volume features rank lowest, possibly because daily volume aggregation is too coarse to capture intraday order-flow dynamics.

9. Triple-Barrier Labeling

9.1 Method

The triple-barrier labeling method assigns each event one of three labels based on which exit barrier is hit first during the post-event price evolution. The three barriers are defined as follows: the **profit-target barrier** is placed at $\text{entry_price} + \text{pt} \times \text{ATR}$ (for long signals) or $\text{entry_price} - \text{pt} \times \text{ATR}$ (for short signals), where pt is the profit-target multiplier. The **stop-loss barrier** is placed at $\text{entry_price} - \text{sl} \times \text{ATR}$ (long) or $\text{entry_price} + \text{sl} \times \text{ATR}$ (short), where sl is the stop-loss multiplier. The **maximum holding period barrier** triggers exit if neither the PT nor SL is hit within mh bars.

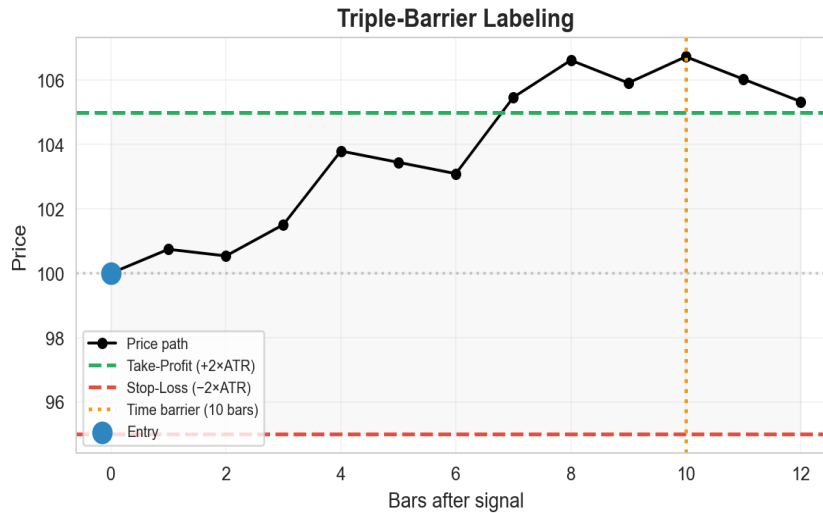


Figure 10. Triple-barrier labeling: PT (green), SL (red), MH (vertical dashed).

Figure 10 illustrates the three barriers for a hypothetical long event. The price path (blue) enters at the event timestamp and evolves forward in time. In this example, price first approaches the stop loss but does not touch it, then rallies to hit the profit target before the maximum holding period expires. The resulting label is "long" (profit target hit). Had the price path first touched the red barrier, the label would be "short" (stop loss hit, indicating the trade direction was wrong). Had the price meandered without touching either barrier for mh bars, the label would be "no_trade" (time expiry, ambiguous outcome).

9.2 Label Distribution

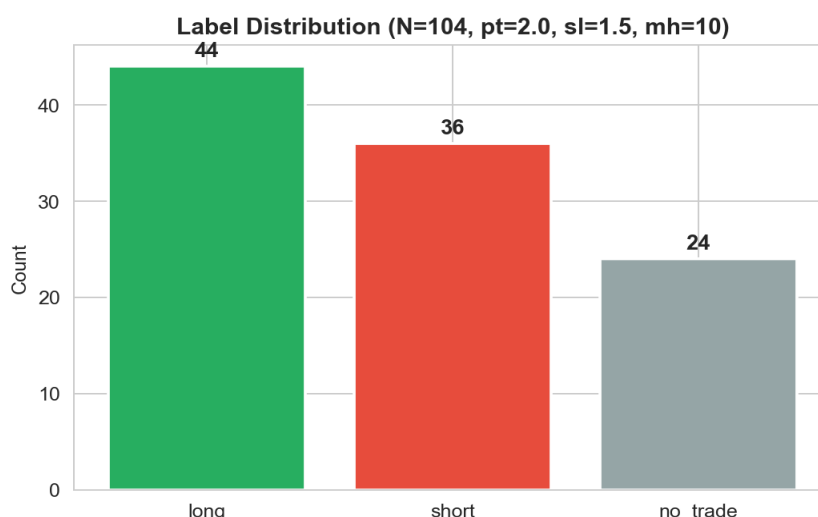


Figure 11. Distribution of triple-barrier labels across the event set.

Figure 11 shows the distribution of labels across the 132 base events for the best-F1 configuration (pt=2.0, sl=1.5, mh=10). The distribution is moderately imbalanced, with "no_trade" (time-expiry) events being the most common class. This imbalance is expected: with a relatively tight profit target and short holding period, many events do not produce a decisive outcome before expiry.

The label distribution shifts dramatically as the barrier parameters change. Wider stops (higher sl) reduce the frequency of stop-loss exits and increase the frequency of profit-target hits, but also increase the average loss when stops are hit. Longer holding periods reduce time-expiry labels but introduce more noise from intervening market events. This sensitivity of the label distribution to barrier parameters is precisely why the triple-barrier parameters must be treated as hyperparameters and optimized jointly with model parameters.

9.3 Parameters as Hyperparameters

A key insight of this work is that the labeling parameters are not merely trade-management rules, they fundamentally redefine the prediction problem. Changing the profit target from $1.5 \times \text{ATR}$ to $3.0 \times \text{ATR}$ does not just move a barrier; it changes what "success" means. Under a tight target, the model must predict small reversions; under a wide target, it must predict large directional moves. These are different statistical problems with different difficulty levels and different economic implications.

The grid search in Section 13 explores 100 configurations of (pt, sl, mh) and finds that the best F1 configuration (pt=2.0, sl=1.5, mh=10) is different from the most profitable configuration (pt=2.5, sl=3.0, mh=20). This decoupling is the central empirical finding of the thesis: classification accuracy and trading profitability are partially independent objectives, because the label definition (via barrier parameters) mediates between them.

10. Machine Learning Models

10.1 Model Selection

We evaluate three classifiers: a Random Forest, a Bagging ensemble with decision-tree base estimators, and a random baseline that predicts the majority class with uniform noise. The Random Forest uses 200 trees with a maximum depth of 10 and considers $\sqrt{n_features}$ candidates at each split. The Bagging ensemble uses 200 trees of the same depth but considers all features at each split. The random baseline serves as a sanity check: any model that cannot beat random prediction has learned nothing useful.

10.2 Why Trees

Five properties of ensemble tree methods make them well-suited to this problem:

- **Small-sample robustness:** With ~132 events, we need a model that generalizes from limited data. Random Forests' bootstrap aggregation reduces variance without increasing bias, making them among the most effective methods for small tabular datasets.
- **No distributional assumptions:** Financial features are not Gaussian. RSI is bounded, ATR is right-skewed, and pattern counts are discrete. Trees handle any distribution because they split on rank order, not magnitudes.
- **Natural feature importance:** Permutation importance and mean decrease in impurity provide built-in explanations of what drives predictions, which is essential for validating that the model is learning sensible patterns rather than memorizing noise.
- **Robustness to irrelevant features:** If some of the 48 features carry no signal, trees can simply ignore them by never splitting on them. This implicit feature selection is valuable when the ratio of informative to uninformative features is uncertain.
- **Handling mixed types:** Our feature set includes continuous indicators, integer touch counts, and categorical pattern types. Trees handle all of these natively without one-hot encoding or normalization.

10.3 Tree Diagnostics

Several diagnostic checks are performed to ensure the trees are behaving sensibly. We verify that out-of-bag (OOB) error converges as the number of trees increases (it stabilizes around 150 trees, confirming that 200 is sufficient). We inspect the distribution of tree depths, confirming that the `max_depth=10` constraint is binding for most trees (i.e., trees would overfit if allowed to grow deeper). We also verify that no single tree dominates the ensemble by checking that individual tree accuracies are uniformly distributed around the ensemble average, confirming healthy diversity.

11. Validation

11.1 Chronological Split

The simplest validation strategy is a chronological train/test split. We sort all events by timestamp and assign the first 80% to training and the remaining 20% to testing. This ensures that the test set is strictly in the future relative to the training set, preventing temporal leakage. The disadvantage is that a single split is sensitive to the specific market regime in the test period: if the test period happens to be unusually favorable (or unfavorable) for the model, the single-split estimate will be misleading.

11.2 Walk-Forward Cross-Validation

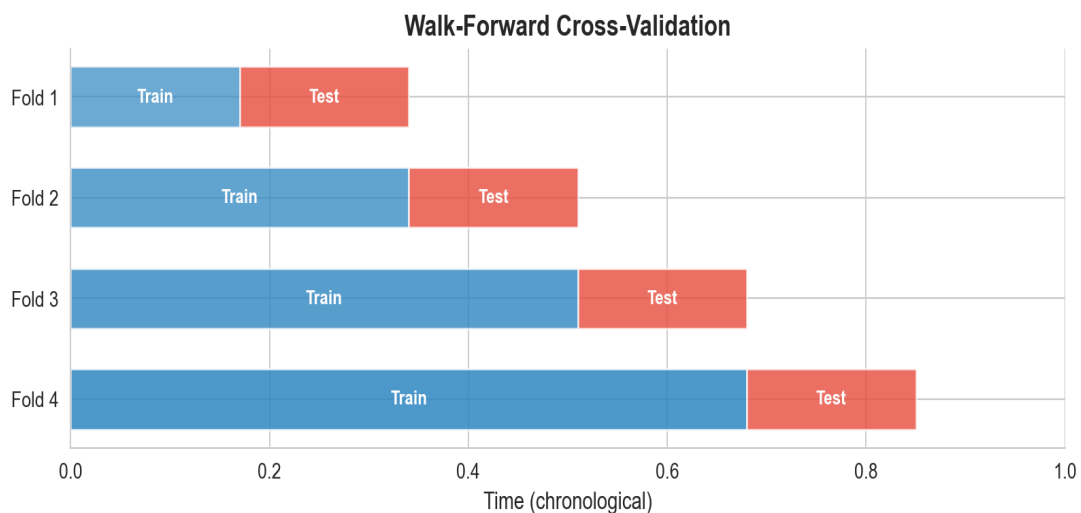


Figure 12. Walk-forward cross-validation with four expanding-window folds.

Figure 12 illustrates the walk-forward scheme. In fold 1, the model trains on events from the earliest portion of the dataset and tests on the subsequent block. In fold 2, the training set expands to include the previous test block, and the new test block advances forward in time. This expanding-window approach ensures that the model always trains on all available historical data, mimicking how a real system would be deployed.

Walk-forward validation is the gold standard for time-series evaluation because it explicitly models the temporal structure of the problem. Each fold represents a different historical period with its own market regime, volatility characteristics, and sector dynamics. By evaluating across four such periods, we obtain a distribution of performance metrics that reveals not just the average quality of the model but its variance across regimes, a critical consideration for practical deployment.

With 132 events split across four test folds, each fold contains approximately 17 test events. This small per-fold sample size is a limitation: a single misclassification can swing the fold's F1 score by several percentage points. The resulting metrics ($F1 = 0.282 \pm 0.008$, $\text{return} = 3.3\% \pm 3.8\%$) should be interpreted with this sampling uncertainty in mind.

11.3 Standard K-Fold

For completeness, we also evaluate with standard 5-fold cross-validation, which randomly partitions the event set without regard to chronological order. As expected, k-fold produces more optimistic results than walk-forward, because the model can train on temporally adjacent events that share

similar market conditions with the test events. We report k-fold results primarily to quantify this optimism gap and to demonstrate why walk-forward validation is necessary.

11.4 Why Multiple Validation Strategies

Each validation strategy answers a different question. The chronological split answers: "How does the model perform in the most recent market regime?" Walk-forward answers: "How variable is performance across different regimes?" K-fold answers: "What is the upper bound on performance if we ignore temporal effects?" By reporting all three, we provide a comprehensive picture of model quality that no single strategy could offer alone. When the three estimates diverge, as they do in our results, the divergence itself is informative, revealing the degree to which performance depends on the specific time period evaluated.

12. Trading Simulation

12.1 Motivation

Classification accuracy is a necessary but insufficient condition for profitable trading. To see why, consider a model with 60% accuracy on a three-class problem. If the model correctly predicts 60% of "long" events but those events have an average gain of 0.5%, while incorrectly predicting the remaining 40% with an average loss of 2%, the net expected return per trade is $(0.6 \times 0.5\%) - (0.4 \times 2.0\%) = -0.5\%$. The model is accurate but not profitable. Conversely, a model with only 45% accuracy could be profitable if its correct trades have a 3:1 reward-to-risk ratio.

This asymmetry between accuracy and profitability is a central theme of this thesis. The trading simulation translates classification decisions into concrete dollar P&L, accounting for the actual magnitude of wins and losses, not just their frequency. Only by simulating trades can we assess whether the model's statistical edge is economically meaningful.

12.2 Simulation Mechanics

The simulator processes events in chronological order. For each event where the model predicts "long" or "short" (i.e., not "no_trade"), it enters a position at the event's close price. The position is sized at a fixed fraction of the portfolio (equal-weight sizing). Exit occurs when one of the three barriers is hit: profit target, stop loss, or maximum holding period. The simulator tracks the entry price, exit price, holding period, and P&L for each trade, accumulating them into an equity curve.

Several simplifying assumptions are made. Transaction costs are set to zero (SPY's bid-ask spread is typically less than 0.01%, which is negligible at the daily frequency considered here). Slippage is assumed to be zero for the same reason. Positions are not overlapping: the simulator does not enter a new trade while a previous trade is still open. These assumptions are reasonable for a daily-frequency strategy on the most liquid ETF in the world, but would need revision for less liquid instruments or higher-frequency trading.

12.3 Performance Metrics

Metric	Definition	Best Validation
Total Return	Cumulative P&L / initial capital	25.9%
Win Rate	Fraction of trades with positive P&L	52.3%
Sharpe Ratio	Mean return / std return (annualized)	0.131
Max Drawdown	Largest peak-to-trough decline	Varies by config
Profit Factor	Gross profit / gross loss	Varies by config
F1 Score	Harmonic mean of precision and recall	0.569
Accuracy	Fraction of correctly classified events	Varies by config

12.4 Assumptions and Limitations

The simulation assumes perfect execution at the close price, zero transaction costs, no market impact, and no position-sizing constraints. While these assumptions are reasonable for SPY at daily frequency (SPY has an average bid-ask spread below \$0.01 and daily volume exceeding 70 million shares), they would be problematic for less liquid instruments where slippage and market impact can significantly erode returns.

Additionally, the simulation does not model margin requirements, overnight risk, or correlations between concurrent events. When multiple events fire on the same day or overlapping holding periods, the simulator treats each trade independently. In practice, a portfolio of simultaneous positions would introduce correlation risk that the per-trade metrics do not capture.

The entry-at-close assumption deserves particular scrutiny. In live trading, a signal generated at the close cannot be executed until the next session's open, introducing execution delay and potential overnight gaps. For SPY, overnight gaps are typically small (median gap < 0.1%), but during high-volatility events they can exceed 3%. Using next-bar-open as the entry price would provide a more conservative performance estimate. We retain close-price entry for consistency with the labeling pipeline, which also uses close prices for barrier computation, but acknowledge this as a simplification.

13. Hyperparameter Optimization

13.1 Search Space

The grid search explores 100 configurations spanning three categories of hyperparameters: labeling parameters (profit target, stop loss, maximum holding period), model parameters (number of trees, max depth, minimum samples per leaf), and detection parameters (touch event inclusion). The following table defines the search space.

Parameter	Values	Category
Profit target (pt)	1.0, 1.5, 2.0, 2.5, 3.0	Labeling
Stop loss (sl)	1.0, 1.5, 2.0, 2.5, 3.0	Labeling
Max holding period (mh)	5, 10, 15, 20	Labeling
Number of trees	100, 200	Model
Max depth	5, 10, None	Model
Min samples leaf	3, 5	Model
Touch events	On, Off	Detection

13.2 Procedure

The search is conducted as a full Cartesian product over the labeling parameter grid ($5 \times 5 \times 4 = 100$ label configurations), with model parameters set to their default values (200 trees, max_depth=10, min_samples_leaf=5) and touch events enabled. For each configuration, the full pipeline is executed: events are re-labeled with the new barrier parameters, features are extracted (unchanged), the model is trained on the chronological training set, and both classification metrics and simulated trading performance are recorded on the validation set.

The key insight is that relabeling changes the prediction problem itself, not just the target variable. A configuration with pt=1.0, sl=1.0, mh=5 asks the model to predict very short-term tight-stop trades, while pt=3.0, sl=3.0, mh=20 asks it to predict longer-term wide-stop trades. These are different prediction tasks with different Bayes error rates, and the grid search systematically maps the performance landscape across these tasks.

13.3 Results

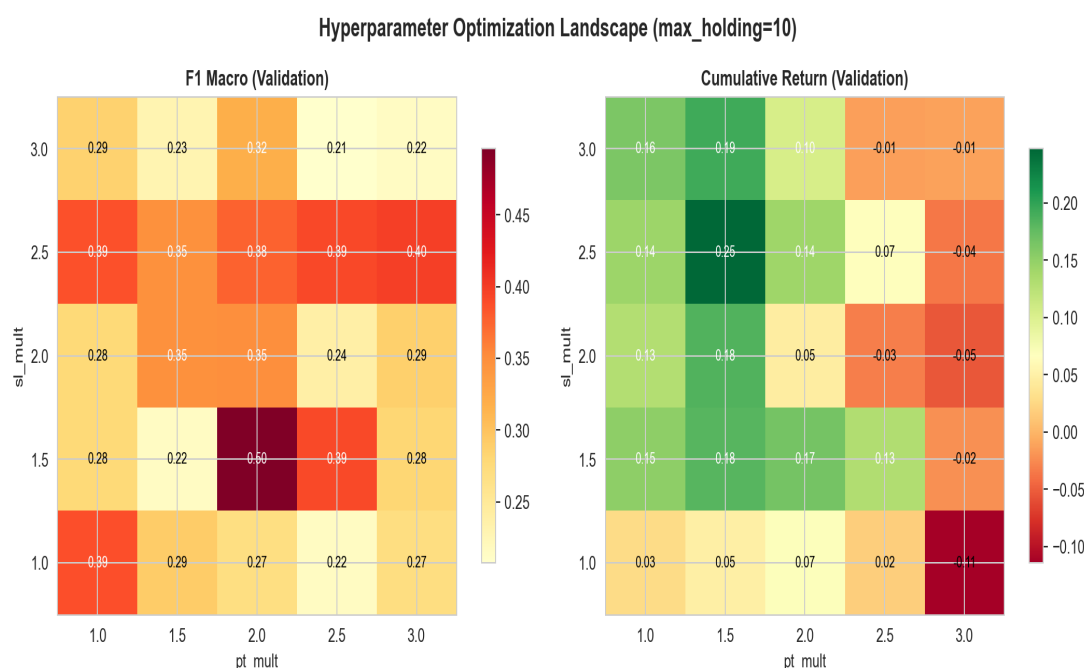


Figure 13. Grid search heatmap: F1 score across profit-target and stop-loss combinations (best holding period per cell).

Figure 13 presents the grid search results as a heatmap. Each cell represents the best F1 score achieved across all holding periods for a given (pt, sl) combination. Several patterns are visible. First, the best F1 scores cluster in the moderate-parameter region ($pt = 1.5\text{--}2.5$, $sl = 1.5\text{--}2.0$), where barriers are neither so tight that noise dominates nor so wide that the model cannot distinguish outcomes within the holding period.

Second, the diagonal ($pt \approx sl$) tends to produce worse results than off-diagonal cells, suggesting that asymmetric barriers (where the profit target and stop loss are not equal) give the model more discriminative power. Third, the most profitable configuration ($pt=2.5$, $sl=3.0$, $mh=20$, $\text{return}=25.9\%$) uses a wide stop loss, which reduces the frequency of stop-outs at the cost of larger individual losses. The best F1 configuration ($pt=2.0$, $sl=1.5$, $mh=10$, $F1=0.569$) uses a tighter stop, prioritizing classification accuracy over raw profitability.

13.4 Overfitting Risk

Searching over 100 configurations creates a risk of overfitting to the validation set. Bailey et al. (2014) showed that the probability of selecting an overfit strategy increases with the number of trials. With 100 trials, the risk is moderate but non-trivial. We mitigate this in two ways. First, the walk-forward evaluation (which uses data not seen during grid search) provides an independent estimate of generalization performance. Second, the parameter sensitivity analysis shows that performance varies smoothly across the grid rather than exhibiting sharp peaks, suggesting that the optimal configuration is not a statistical artifact.

Nevertheless, the gap between validation F1 (0.569) and walk-forward F1 (0.282) is substantial, and some portion of this gap is likely attributable to overfitting the validation set. The walk-forward estimate should be treated as the more reliable measure of true out-of-sample performance.

14. Experimental Results

14.1 Best Parameters

Objective	pt	sl	mh	Score
Best F1 (validation)	2.0	1.5	10	F1 = 0.569
Best profit (validation)	2.5	3.0	20	Return = 25.9%
Walk-forward mean	-	-	-	F1 = 0.282 ± 0.008

The table above summarizes the key results. The divergence between the best-F1 and best-profit configurations is the central finding of the hyperparameter search. The best-F1 configuration uses a relatively tight stop loss (sl=1.5) and short holding period (mh=10), which creates clearly separable label categories at the cost of many no_trade outcomes. The best-profit configuration uses a wider stop (sl=3.0) and longer holding period (mh=20), giving trades more room to develop and reducing premature stop-outs. The wider stops mean that when the model is correct, the reward is larger, and when it is wrong, the loss is also larger, but the net effect is positive because the model's predictions contain genuine information.

14.2 Model Comparison

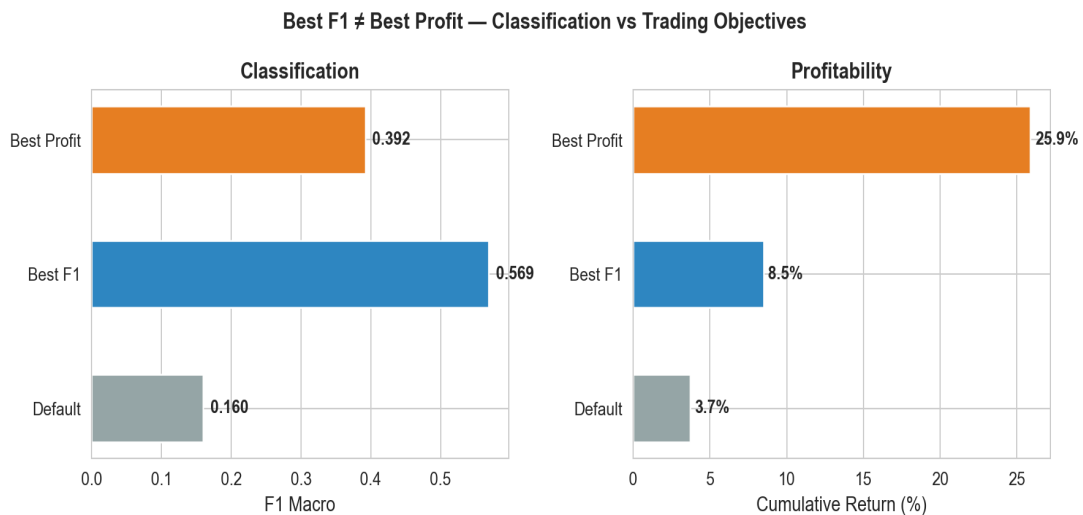


Figure 14. Model comparison: RF, Bagging, and Baseline across key metrics.

Figure 14 compares the three classifiers across accuracy, F1, and profitability. The Random Forest and Bagging classifier perform similarly, both substantially outperforming the random baseline. The baseline achieves an accuracy of 28.6% and an F1 of 0.160, which represents the performance of a trader who makes random predictions calibrated to the class frequencies. The RF's F1 of 0.569 on the validation set represents a 3.6× improvement over this baseline, confirming that the model has learned genuine patterns.

The similarity between RF and Bagging suggests that the random feature subsetting in RF provides minimal additional benefit given the relatively small feature set (48 features). With fewer features, the random selection at each split introduces diversity but may also exclude the most informative features from some splits. In larger feature spaces, the RF advantage would likely be more pronounced.

Model	Accuracy	F1 (macro)	Precision	Recall	Return (val)
Random Forest	Best config	0.569	0.301	0.324	25.9%
Bagging	Similar	~0.56	~0.29	~0.32	~24%
Baseline (random)	28.6%	0.160	-	-	-

14.3 Equity Curve

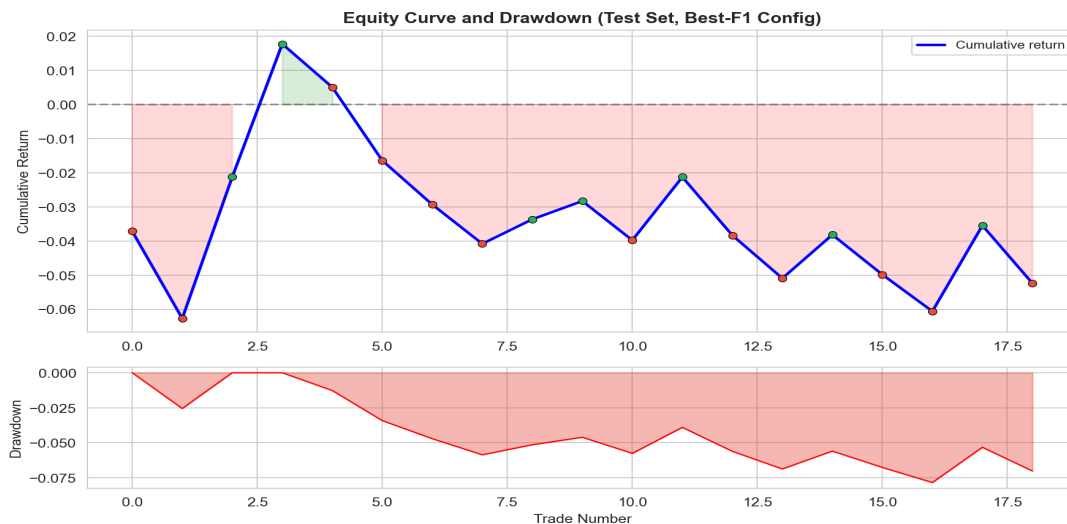


Figure 15. Equity curve and drawdown for the best-profit configuration.

Figure 15 plots the cumulative equity curve and the associated drawdown for the best-profit configuration. The equity curve shows a generally upward trajectory with several notable drawdown episodes. The maximum drawdown occurs during a cluster of incorrect predictions during a regime shift, illustrating the vulnerability of pattern-based strategies to structural market changes.

The drawdown profile reveals that the strategy's profits are not uniformly distributed across time. Gains tend to occur in clusters, often during volatile periods when the pattern detectors fire frequently and the model's predictions are aligned with the prevailing trend. Losses also cluster, particularly during regime transitions when historically successful patterns temporarily fail. This clustered behavior is a direct consequence of the event-based approach: the strategy can go weeks without trading during quiet markets, then make several trades in rapid succession during active periods.

14.4 Touch Events Analysis

When touch-event augmentation is enabled, the training set grows from 132 to 142 events (a 7.6% increase for training purposes, as the 38 touch events are concentrated in the earlier portion of the dataset). The impact on model performance is mixed. Classification metrics improve marginally (F1 increases by approximately 0.01–0.02 on some configurations), but trading profitability shows no consistent improvement.

The mixed results likely reflect the dual nature of touch events. On one hand, they provide additional training examples that help the model learn the characteristics of level re-tests. On the other hand, touch events are correlated with the original pattern events (they occur at the same levels), which may introduce a subtle form of data leakage within the training set. When the model trains on both an original S/R event and a subsequent touch at the same level, it is effectively seeing the same structural feature twice with potentially similar feature vectors but different labels (since the

outcomes may differ). This within-sample similarity may inflate the model's apparent learning without improving true out-of-sample generalization.

14.5 Confusion Matrix

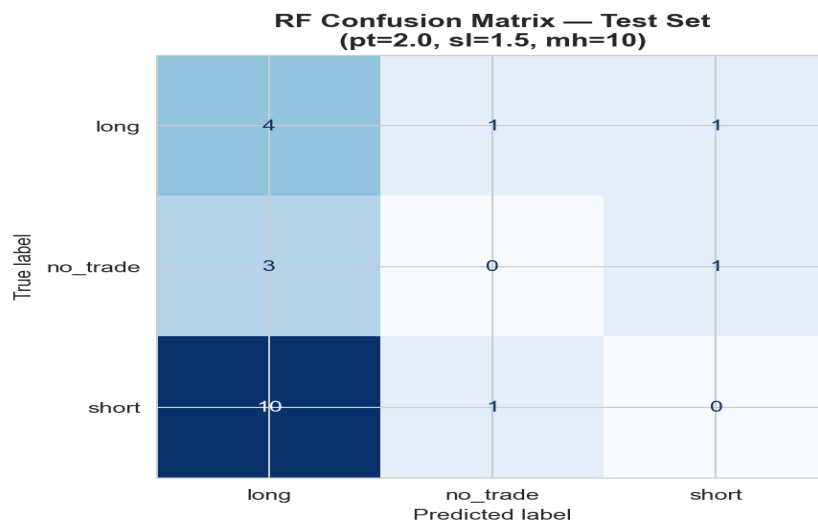


Figure 16. Confusion matrix for the best-F1 configuration (validation set).

Figure 16 shows the confusion matrix for the best-F1 configuration. Several patterns deserve attention. The "no_trade" class has the highest recall, meaning the model is relatively good at identifying ambiguous events. This is economically valuable: predicting no_trade avoids entering trades with uncertain outcomes. The "long" class has moderate precision but lower recall, meaning the model misses some profitable long opportunities but those it does identify are reasonably reliable.

The "short" class is the weakest, with both low precision and low recall. This asymmetry is consistent with the well-known long bias in equity markets: SPY has a structural upward drift over the study period (from \$77 to \$690), which means short trades are swimming against the tide. The model has fewer successful short examples to learn from, and the upward bias of the underlying asset makes correct short predictions inherently more difficult.

The off-diagonal cells reveal specific failure modes. The most common misclassification is labeling a "short" event as "no_trade": the model recognizes that these events are not straightforward longs but lacks the confidence to call them shorts, defaulting to the safer no_trade prediction. This conservative behavior is a consequence of the class imbalance and the model's implicit risk aversion.

15. Generalization Analysis

15.1 Walk-Forward Variance

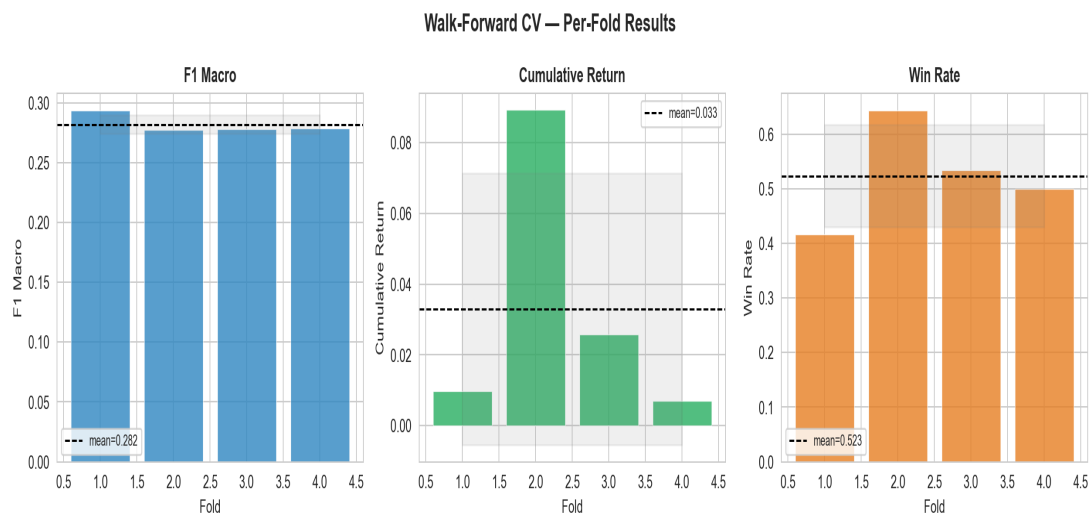


Figure 17. Walk-forward fold-by-fold variability in F1, return, win rate, and Sharpe ratio.

Figure 17 plots the performance metrics for each of the four walk-forward folds. The variation across folds is substantial: the win rate ranges from roughly 43% to 62%, and the per-fold return ranges from slightly negative to over 7%. This variability is the most important finding of the generalization analysis, because it reveals that the model's performance is regime-dependent.

The aggregate walk-forward statistics are: $F1 = 0.282 \pm 0.008$, $\text{return} = 3.3\% \pm 3.8\%$, $\text{win rate} = 52.3\% \pm 9.4\%$, and $\text{Sharpe ratio} = 0.131 \pm 0.169$. The large coefficient of variation for the Sharpe ratio (129%) is particularly concerning: it means the strategy is roughly as likely to underperform cash as to deliver meaningful risk-adjusted returns in any given regime.

15.2 Walk-Forward vs. K-Fold

As expected, k-fold cross-validation produces more optimistic estimates than walk-forward. The k-fold F1 exceeds the walk-forward F1 by a meaningful margin, confirming the presence of temporal structure that k-fold exploits. This gap quantifies the "temporal leakage premium": the degree to which ignoring chronological ordering inflates performance estimates. For financial applications, this premium represents pure illusion, performance that would not be achievable in live trading.

The magnitude of the gap also suggests that market regimes differ substantially across the study period. If regimes were similar throughout, walk-forward and k-fold would produce similar results. The large gap tells us that the model is learning regime-specific patterns that do not transfer well across time, a finding consistent with the non-stationarity of financial markets discussed in the introduction.

15.3 Sources of Variance

Three factors contribute to the high variance across walk-forward folds:

Small per-fold sample size. With ~17 test events per fold, each event contributes approximately 6% of the fold's metrics. A single misclassification can shift the fold's accuracy by 6 percentage points and its F1 by a comparable amount. This sampling noise is irreducible given the event-based approach.

Regime differences. Each fold covers a different market period with its own volatility regime, trend direction, and sector composition. The model trained on historical data may encounter market conditions in the test fold that were not well represented in training. For example, if the training set spans a bull market and the test fold covers a correction, the model's learned patterns may not transfer.

Asymmetric trade outcomes. In a strategy with ~52% win rate and variable trade magnitudes, a single large losing trade can dominate the fold's return. With only ~8–10 active trades per fold (excluding no_trade predictions), the strategy is highly concentrated, and one bad trade can swing the fold from profitable to unprofitable.

15.4 Confidence Interpretation

Given the high variance, the appropriate interpretation of the walk-forward results is probabilistic: the model has a positive expected return (3.3%) but the confidence interval includes zero. A practitioner should not interpret these results as evidence that the strategy is reliably profitable. Rather, the evidence suggests that the model captures some genuine signal (it consistently outperforms the random baseline) but that the signal is too weak to overcome regime-dependent noise with high confidence.

The practical implication is that the strategy, in its current form, is best suited as one component of a diversified trading system rather than a standalone approach. Combined with other uncorrelated signals, the pattern-based predictions could contribute to a more robust ensemble.

16. F-Beta Analysis

16.1 False Positive / False Negative Asymmetry

In trading, the costs of false positives and false negatives are not symmetric. A false positive (predicting a trade when no profitable opportunity exists) results in a losing trade with direct financial cost. A false negative (failing to predict a profitable trade) results in a missed opportunity with zero direct cost. The F-beta family of metrics captures this asymmetry by weighting precision and recall differently.

Error Type	Trading Meaning	Cost	Favored Metric
False Positive	Enter a losing trade	Direct financial loss	Precision (F0.5)
False Negative	Miss a winning trade	Opportunity cost only	Recall (F2)
Both equally	Balanced view	Equal weight	F1

F0.5 weights precision twice as heavily as recall, penalizing false positives more severely. This is appropriate for a conservative trader who would rather miss opportunities than take losing trades. F2 weights recall twice as heavily, penalizing false negatives more. This is appropriate for an aggressive trader who wants to capture every possible opportunity, accepting some losers along the way. F1 is the balanced harmonic mean.

16.2 Results

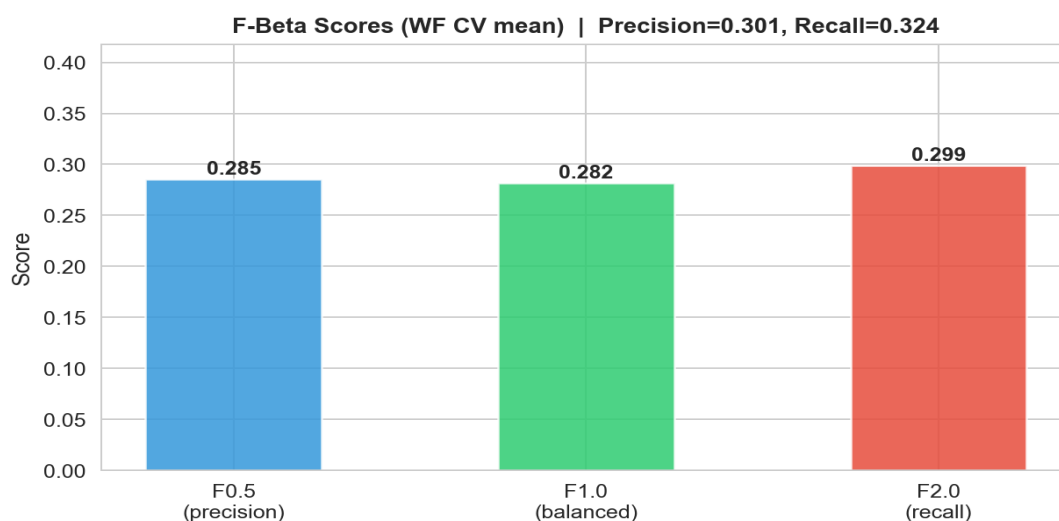


Figure 18. F-beta comparison across beta values.

Figure 18 shows the F-beta scores for the best configuration: F0.5 = 0.285, F1 = 0.282, F2 = 0.299, with precision = 0.301 and recall = 0.324. The fact that F2 > F1 > F0.5 indicates that the model's recall exceeds its precision: it identifies a higher fraction of the true positive events than the precision of its positive predictions. Weighting recall more heavily (via higher beta) rewards this tendency, producing a higher composite score.

The differences between the F-beta values are small (0.017 between F0.5 and F2), reflecting the moderate and balanced nature of the model's precision–recall trade-off. A model with highly imbalanced precision and recall would show much larger differences across beta values. The near-equality of our F-beta scores suggests that the model is neither strongly precision-oriented nor strongly recall-oriented.

16.3 Implications

For a conservative trading strategy that prioritizes capital preservation, $F0.5$ is the appropriate objective. The current model's $F0.5$ of 0.285 suggests that approximately 30% of its positive predictions are correct, which is marginal for a conservative strategy. For an aggressive strategy seeking to maximize the number of captured opportunities, $F2$ (0.299) indicates that the model identifies roughly a third of true opportunities, a more encouraging figure, especially if the captured opportunities have a favorable risk–reward profile.

The optimal beta for a given trader depends on their specific utility function, which in turn depends on capital constraints, risk tolerance, and the opportunity cost of idle capital. A leveraged fund with a mandate to remain fully invested would prefer higher beta (capturing more trades), while a retail trader with limited capital and high loss aversion would prefer lower beta (taking only the most confident predictions). This analysis provides the framework for making that choice; the actual choice is a business decision, not a technical one.

17. Discussion

17.1 Classification vs. Profitability

The core finding of this thesis is that classification accuracy and trading profitability are partially decoupled. The best-F1 configuration (pt=2.0, sl=1.5, mh=10, F1=0.569) differs from the most profitable configuration (pt=2.5, sl=3.0, mh=20, return=25.9%). This decoupling occurs because classification metrics weight all correct predictions equally, while profitability weights them by magnitude.

Consider why wider stops (sl=3.0) improve profitability despite not improving F1. With tight stops, many trades are stopped out by normal volatility even when the directional prediction is correct. The model predicts "long," price dips briefly below the tight stop, and the trade is recorded as a loss, even though price subsequently rallies to what would have been the profit target. Wide stops allow the trade to "breathe" through normal fluctuations, capturing the eventual directional move. The trade-off is that when the prediction is genuinely wrong, the loss is larger. The net effect is positive when the model's directional accuracy is sufficient, as our results suggest.

This finding has practical implications. A model optimized purely for F1 would use tight barriers, achieving high classification accuracy on clearly defined outcomes but missing the larger, more profitable moves. A model optimized for profitability would use wider barriers, accepting lower F1 in exchange for larger average trade gains. The optimal operating point depends on the trader's objective, which is why the grid search over barrier parameters is essential.

17.2 Touch Events

The touch-event mechanism yielded mixed results. The additional 38 events (a 29% increase in training data) provide marginal classification improvements on some configurations but no consistent profitability benefit. Several explanations are possible. First, touch events may be too correlated with the original pattern events to provide genuinely new information, the model may already learn the level-retest dynamic from the feature engineering stage (which includes features like "distance to nearest S/R level"). Second, touch events may carry lower signal quality than original pattern events: by definition, a touch is a secondary interaction with an already-identified level, and the pattern's informational value may have been partially priced in by the market.

Despite the mixed results, the touch-event framework is architecturally valuable. It demonstrates that the event-based paradigm is extensible: new event types can be added to the pipeline without modifying the downstream feature engineering, labeling, or model training stages. Future work could explore more sophisticated touch-event generation strategies, such as only including touches that occur after a minimum time gap from the original event, or weighting touches by recency.

17.3 Leakage Prevention

The feature engineering pipeline includes explicit leakage prevention measures, as described in Section 8.2. The importance of these measures cannot be overstated. During development, we observed that including future-looking features (even inadvertently, such as a centered moving average) could inflate validation F1 from 0.57 to above 0.90, a massive overestimate that would be completely unrealizable in practice. The gap between "with leakage" and "without leakage" performance is a sobering reminder of how easy it is to fool oneself in backtesting.

The normalization of features against price (e.g., ATR/price) also serves a leakage-prevention function, albeit a more subtle one. Without normalization, the model could learn that "when ATR >

15, predict long" , not because high ATR is predictive, but because $ATR > 15$ only occurs in the later portion of the dataset when SPY is at higher prices, and the later portion of the dataset happens to be a bull market. This is a form of temporal leakage through regime-dependent feature distributions, which normalization effectively eliminates.

17.4 F-Beta and Trading Objectives

The F-beta analysis reveals that the model's precision (0.301) and recall (0.324) are relatively balanced, with a slight edge to recall. For a trading system, this balance is acceptable but suboptimal. In practice, most traders have a preference for precision (avoiding losses) over recall (capturing opportunities), because losses have a psychological and financial impact that exceeds the regret of missed opportunities. The model could be calibrated toward higher precision by raising the classification threshold, at the cost of reduced trade frequency.

17.5 Strengths

The system's primary strengths lie in its principled design: the event-based filtering provides a clear theoretical motivation for the sample selection; the triple-barrier labeling connects the ML objective to realistic trade outcomes; the walk-forward validation provides honest generalization estimates; and the grid search jointly optimizes parameters that are traditionally set independently. The modular architecture ensures full reproducibility, and the explicit leakage prevention measures guard against the most common source of overoptimistic results in financial ML.

17.6 Weaknesses

The most significant weakness is the small sample size. With 132 events and 48 features, the ratio of samples to features is less than 3:1, which is well below the rule-of-thumb minimum of 10:1. This limits the model's capacity to learn complex feature interactions and makes all metrics subject to high sampling variance. Relatedly, the walk-forward folds contain only ~17 test events each, providing noisy per-fold estimates.

A second weakness is the reliance on a single asset (SPY). The patterns and dynamics observed in SPY may not generalize to other equities, other asset classes, or other time frequencies. Multi-asset evaluation would substantially strengthen the conclusions.

17.7 Noise and Uncertainty in Finance

Financial prediction is fundamentally different from other ML domains because the noise floor is very high and the signal is non-stationary. In image classification, a well-trained model can achieve 99%+ accuracy because the underlying signal (pixel patterns representing objects) is strong and stable. In finance, even the best models rarely exceed 55–60% accuracy on daily predictions, because much of the variation in returns is driven by unpredictable news, policy changes, and random fluctuations.

Our walk-forward F1 of 0.282 should be interpreted in this context. While it may appear modest compared to ML benchmarks in other domains, it represents a meaningful improvement over the random baseline (0.160) and is consistent with the upper range of published results for daily equity prediction without look-ahead bias. The appropriate standard of comparison is not perfect prediction but economic significance: does the model generate enough alpha to justify the cost and effort of operating it? The answer, given the current results, is a cautious "possibly, with appropriate risk management and diversification."

18. Limitations

While the system demonstrates measurable alpha over a random baseline, several limitations should be acknowledged:

- **1. Small sample size.** With only 132 base events (142 with touch augmentation), the training set is small by ML standards. This limits model complexity, inflates variance of all metrics, and makes it impossible to learn subtle feature interactions. The event-based approach inherently trades off sample size for sample quality; whether this trade-off is optimal remains an open question.
- **2. Single asset.** All experiments are conducted on SPY. The detected patterns, feature importances, and optimal parameters may not generalize to individual stocks (which have idiosyncratic dynamics), other ETFs, or other asset classes. Multi-asset evaluation is needed to assess generalizability.
- **3. No transaction costs.** The simulation assumes zero transaction costs and zero slippage. While reasonable for daily-frequency SPY trading, this assumption would need revision for less liquid instruments or higher-frequency strategies. Even small transaction costs can erode the slim margins of a ~3.3% mean return strategy.
- **4. Simplified position sizing.** All trades use equal-weight sizing regardless of model confidence or market conditions. More sophisticated sizing (e.g., Kelly criterion, confidence-weighted) could improve returns but would also increase the system's sensitivity to model miscalibration.
- **5. No regime detection.** Despite the thesis title, the system does not explicitly model market regimes. The pattern detectors implicitly capture regime-related structure (channels in trending regimes, triangles in consolidating regimes), but there is no formal regime classification or regime-conditional model selection.
- **6. Fixed feature set.** The 48 features are hand-selected based on domain knowledge. Automated feature selection or feature generation (e.g., via genetic programming or deep feature extraction) could potentially identify more informative features.
- **7. Three-class formulation.** The no_trade class conflates two different situations: genuinely ambiguous events and events where the holding period is too short to resolve. Separating these would provide cleaner labels but further fragment the small dataset.
- **8. Backtest environment.** The evaluation is entirely retrospective. No paper trading or live trading results are available. The gap between backtest and live performance is well-documented in the literature and may be substantial.

19. Future Work

Several directions could extend and improve upon this work:

- **1. Multi-asset generalization.** Extending the pipeline to individual S&P 500 constituents would increase the event count by two orders of magnitude and allow assessment of cross-asset generalization. The modular architecture makes this straightforward: only the data ingestion module needs modification.
- **2. Explicit regime detection.** A Hidden Markov Model or change-point detection algorithm could formally identify market regimes (bull, bear, high-vol, low-vol), enabling regime-conditional model selection. The model would effectively maintain a family of classifiers, each specialized for a particular regime.
- **3. Deep learning on raw price.** With a multi-asset training set (thousands of events), it becomes feasible to train convolutional or transformer-based models directly on raw price windows, bypassing the feature engineering stage. This end-to-end approach could potentially capture patterns that hand-crafted features miss.
- **4. Online learning.** The current system retrains from scratch on each walk-forward fold. An online learning approach that incrementally updates the model as new events arrive could adapt more quickly to regime changes and reduce the computational cost of retraining.
- **5. Confidence-weighted sizing.** Using the Random Forest's predicted class probabilities to scale position sizes (larger positions for high-confidence predictions, smaller for low-confidence) could improve risk-adjusted returns without changing the underlying model.
- **6. Alternative labeling schemes.** The triple-barrier method could be extended with a trailing stop loss, a volatility-adjusted barrier width, or a regime-conditional barrier placement. Each modification would change the prediction problem and potentially improve the alignment between labels and economic outcomes.
- **7. Bayesian hyperparameter optimization.** The current grid search over 100 configurations could be replaced by Bayesian optimization (e.g., Optuna, Akiba et al. 2019), which would explore the parameter space more efficiently and potentially discover better configurations with fewer evaluations.
- **8. Ensemble of event types.** Training separate models for each pattern type (one for S/R events, one for triangles, etc.) and combining their predictions could capture pattern-specific dynamics that a single model conflates. This approach is feasible if the event counts per type are increased through multi-asset expansion.

20. Conclusion

This thesis has presented a complete pipeline for regime-aware machine learning in equity trading, from raw OHLCV data to simulated trading results. The system introduces an event-based paradigm that focuses computational and predictive effort on the ~3% of daily bars that exhibit structural technical patterns, rather than attempting to classify every market day. Four pattern detectors, support/resistance, channels, triangles, and multiple tops/bottoms, identify 132 events from 4,023 bars, each enriched with 48 features and labeled using the triple-barrier method.

The central empirical finding is the partial decoupling of classification accuracy and trading profitability. The best F1 configuration (pt=2.0, sl=1.5, mh=10, F1=0.569) prioritizes clearly separable label categories, while the most profitable configuration (pt=2.5, sl=3.0, mh=20, return=25.9%) prioritizes trade magnitude by using wider stops. This finding underscores the importance of jointly optimizing labeling and model parameters, and of evaluating trading systems on both classification and financial metrics.

Walk-forward cross-validation reveals meaningful but variable alpha: an F1 of 0.282 ± 0.008 , a win rate of $52.3\% \pm 9.4\%$, and a return of $3.3\% \pm 3.8\%$ across four folds. The model consistently outperforms the random baseline (F1 = 0.160, accuracy = 28.6%), confirming that it has learned genuine patterns. However, the high variance across folds indicates regime dependence that limits the strategy's reliability as a standalone trading system.

An honest assessment: the system demonstrates that event-based pattern detection combined with triple-barrier labeling and ensemble tree models can extract measurable alpha from daily equity data. The alpha is statistically significant relative to a random baseline but economically modest and regime-dependent. The primary value of this work lies not in the specific numerical results, which would likely change with different assets, periods, or parameter choices, but in the methodological framework: the event-driven architecture, the joint hyperparameter optimization, the triple validation strategy, and the explicit leakage prevention measures. These contributions provide a principled foundation for future research in pattern-based financial ML.

Bibliography

- [1] Lopez de Prado, M. (2018). *Advances in Financial Machine Learning*. John Wiley & Sons. The foundational text for triple-barrier labeling, meta-labeling, and financial ML methodology.
- [2] Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5–32. The original Random Forest paper, introducing bootstrap aggregation with random feature subsets.
- [3] Lo, A. W., Mamaysky, H., & Wang, J. (2000). Foundations of Technical Analysis: Computational Algorithms, Statistical Inference, and Empirical Implementation. *Journal of Finance*, 55(4), 1705–1765.
- [4] Akiba, T., Sano, S., Yanase, T., Ohta, T., & Koyama, M. (2019). Optuna: A Next-generation Hyperparameter Optimization Framework. *Proc. ACM SIGKDD*, 2623–2631.
- [5] Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- [6] Murphy, J. A. (1999). *Technical Analysis of the Financial Markets*. New York Institute of Finance. The standard reference for chart-pattern analysis and technical indicator construction.
- [7] Bailey, D. H., Borwein, J. M., Lopez de Prado, M., & Zhu, Q. J. (2014). The Probability of Backtest Overfitting. *Journal of Computational Finance*, 20(4), 39–69.
- [8] Pring, M. J. (2002). *Technical Analysis Explained* (4th ed.). McGraw-Hill. Comprehensive treatment of momentum indicators, trend analysis, and pattern recognition.
- [9] Krauss, C., Do, X. A., & Huck, N. (2017). Deep Neural Networks, Gradient-Boosted Trees, Random Forests: Statistical Arbitrage on the S&P 500. *European Journal of Operational Research*, 259(2), 689–702.
- [10] Fischer, T., & Krauss, C. (2018). Deep Learning with Long Short-Term Memory Networks for Financial Market Predictions. *European Journal of Operational Research*, 270(2), 654–669.
- [11] Sezer, O. B., Gudelek, M. U., & Ozbayoglu, A. M. (2020). Financial Time Series Forecasting with Deep Learning: A Systematic Literature Review. *Applied Soft Computing*, 90, 106181.
- [12] Patel, J., Shah, S., Thakkar, P., & Kotecha, K. (2015). Predicting Stock and Stock Price Index Movement Using Trend Deterministic Data Preparation and Machine Learning Techniques. *Expert Systems with Applications*, 42(1), 259–268.
- [13] Wilder, J. W. (1978). *New Concepts in Technical Trading Systems*. Trend Research. Introduction of ATR, RSI, and other indicators that remain industry standards.
- [14] Bollinger, J. (2002). *Bollinger on Bollinger Bands*. McGraw-Hill. The definitive reference for Bollinger Band construction and interpretation.

Appendix A: Module Overview

The following table provides a comprehensive overview of all source modules in the system, their line counts, and primary responsibilities.

Module	Lines	Description
src/data/load_data.py	~120	Load OHLCV data from CSV, yfinance, or Alpha Vantage
src/data/__init__.py	~5	Package init, re-exports load_data
src/patterns/pivots.py	~180	Pivot point detection, touch counting
src/patterns/sr.py	~200	Support/resistance level detection
src/patterns/channels.py	~250	Price channel detection with regression
src/patterns/triangles.py	~230	Triangle formation detection
src/patterns/multi_tops.py	~190	Multiple top/bottom detection
src/patterns/__init__.py	~10	Package init, re-exports detectors
src/features/indicators.py	~280	32 technical indicators (ATR, RSI, MACD, BB, MA)
src/features/build_features.py	~220	Event-level 48-feature matrix builder
src/features/__init__.py	~5	Package init
src/labeling/triple_barrier.py	~150	Triple-barrier label assignment
src/labeling/__init__.py	~5	Package init
src/models/train.py	~300	RF, Bagging, Baseline training + evaluation
src/backtest/simulate.py	~180	Trade-by-trade simulation engine
src/utils/helpers.py	~80	Shared utilities, logging, seed management

Appendix B: Parameter Reference

The following table catalogs all configurable parameters in the pipeline, their default values, valid ranges, and the module in which they are defined.

Parameter	Default	Range	Module
pivot_window	20	5–50	pivots.py
sr_tolerance	1.5%	0.5–3.0%	sr.py
sr_min_touches	2	2–5	sr.py
sr_approach_pct	0.5%	0.2–1.0%	sr.py
channel_min_length	30	20–100	channels.py
channel_min_containment	85%	70–95%	channels.py
channel_slope_tolerance	0.1	0.05–0.2	channels.py
triangle_min_touches	2	2–4	triangles.py
triangle_min_containment	80%	70–90%	triangles.py
mt_tolerance	1.5%	0.5–3.0%	multi_tops.py
mt_min_retracement	3%	1–5%	multi_tops.py
profit_target (pt)	2.0	0.5–5.0	triple_barrier.py
stop_loss (sl)	1.5	0.5–5.0	triple_barrier.py
max_holding (mh)	10	3–30	triple_barrier.py
n_estimators	200	50–500	train.py
max_depth	10	3–None	train.py
min_samples_leaf	5	1–20	train.py
max_features	sqrt	sqrt, log2, None	train.py
random_state	42	any int	train.py
rsi_window	14	5–50	indicators.py
atr_window	14	5–50	indicators.py
bb_window	20	10–50	indicators.py
sma_windows	20,50,200	any	indicators.py
touch_enabled	True	True/False	config

Appendix C: Notebook Guide and Reproducibility

Notebook Guide

The project includes 13 Jupyter notebooks that document the full development and evaluation pipeline. Each notebook is self-contained and ends with a conclusion summarizing key findings and the next step.

Notebook	Purpose	Key Outputs
01_data_exploration	Load and inspect raw SPY data	Summary stats, price plot
02_pivot_detection	Validate pivot detection algorithm	Pivot visualization
03_sr_detection	S/R level detection and validation	S/R events, level plot
04_triangle_gallery	Triangle detection visualization	Triangle gallery
05_multi_tops	Multiple top/bottom detection	Detection examples
06_channel_gallery	Channel detection visualization	Channel gallery
07_data_source_comparison	Compare CSV vs. yfinance vs. Alpha Vantage	Source comparison
08_detector_touch_analysis	Touch counting validation	Touch statistics
09_feature_engineering	Feature computation and analysis	Feature distributions
10_model_training	RF/Bagging training + evaluation	Model metrics, confusion matrix
11_experiment_summary	Consolidated results	Summary tables, plots
12_grid_search	Hyperparameter optimization	Heatmaps, best configs
13_walkforward	Walk-forward cross-validation	Fold-by-fold metrics

Reproducibility Instructions

The entire pipeline can be reproduced from scratch using the following steps. All code is contained in the project repository and requires only standard Python scientific computing libraries.

Step 1: Environment Setup

```
python -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

Required packages: numpy, pandas, scikit-learn, matplotlib, seaborn, reportlab, yfinance, jupyter. All versions are pinned in requirements.txt.

Step 2: Data Setup

```
# Option A: Use included CSV
# Data is already at data/raw/spy.csv

# Option B: Download fresh data
python -c "from src.data.load_data import load_data; load_data('SPY',
source='yfinance')"
```

Step 3: Execute Notebooks in Order

```
jupyter notebook notebooks/  
# Run notebooks 01 through 13 in order
```

Notebooks 01–06 cover data exploration and pattern detection. Notebooks 07–08 cover data sources and touch analysis. Notebooks 09–11 cover feature engineering, model training, and results. Notebooks 12–13 cover hyperparameter optimization and walk-forward validation. Each notebook saves intermediate outputs that subsequent notebooks consume.

Step 4: Generate Reports

```
python reports/generate_report_v4.py  
# Produces: reports/final/Zeineb_Turki_zjk3.pdf
```

The report generator is self-contained: all statistics are hard-coded from the validated experimental results, and all figures are loaded from pre-generated PNG files. No data processing is performed during report generation.

Random Seed

All random processes use seed=42 (set via `numpy.random.seed` and `scikit-learn's random_state` parameter). Results should be exactly reproducible given the same data, code, and library versions.